Base R: 120 Exercises with Solutions

Vectors · Operators · Functions · Data Frames · Lists · Matrices · Programming · Apply · Strings · Regex · Date/Time · I/O · Error Handling

Outline

1. Vectors and Types (Ex 1–18)
2. Operators and Common Functions (Ex 19–26)
3. Data Frames (Ex 27–40)
4. Lists (Ex 41–52)
5. Matrices (Ex 53–58)
6. Programming (Ex 59–84)
7. Apply Family (Ex 85–96)
8. Strings and Regex (Ex 97–110)
9. Date/Time, I/O, Error Handling, Advanced (Ex 111–120)

Vectors and Types (Ex 1–18)

Ex 1: Logical Indexing

Problem

Let `v <- c(14, -3, 22, 7, -8, 0, 31, 5, -1, 18)`.

- **(a)** Extract all elements strictly between 5 and 25
- **(b)** Replace every negative element with `0` in a copy `w`
- **(c)** How many elements are negative or exceed 20?
- **(d)** Find indices where $|v_i|$ exceeds its rank

Sol 1

```r
v <- c(14, -3, 22, 7, -8, 0, 31, 5, -1, 18)
v[v > 5 & v < 25]                    # (a)
```

```
# [1] 14 22  7 18
```

```r
w <- v; w[w < 0] <- 0; w             # (b)
```

```
#  [1] 14  0 22  7  0  0 31  5  0 18
```

```r
sum(v < 0 | v > 20)                  # (c)
```

```
# [1] 5
```

```r
which(abs(v) > rank(v))              # (d)
```

```
# [1]  1  2  3  4  5  7 10
```

Ex 2: Named Vectors

Problem

Create `stock <- c(AAPL=189.5, MSFT=415.2, GOOG=172.3, AMZN=185.7)`.

- **(a)** Access `GOOG` by name; add `NVDA = 875.4`
- **(b)** Compute percentage change from base 150; round to 1 dp
- **(c)** Return stocks above mean price, sorted descending
- **(d)** Rename all tickers to lowercase

Sol 2

```r
stock <- c(AAPL=189.5, MSFT=415.2, GOOG=172.3, AMZN=185.7)
stock["GOOG"]; stock["NVDA"] <- 875.4             # (a)
```

```
#  GOOG
# 172.3
```

```r
round((stock - 150) / 150 * 100, 1)               # (b)
```

```
#  AAPL  MSFT  GOOG  AMZN  NVDA
#  26.3 176.8  14.9  23.8 483.6
```

```r
sort(stock[stock > mean(stock)], decreasing=TRUE)  # (c)
```

```
#  NVDA  MSFT
# 875.4 415.2
```

```r
names(stock) <- tolower(names(stock)); names(stock)# (d)
```

```
# [1] "aapl" "msft" "goog" "amzn" "nvda"
```

Ex 3: `which`, `which.min`, `which.max`

Problem

Let `x <- c(3, 7, 2, 9, 1, 5, 8, 4, 6, 10)`.

- **(a)** Indices of elements > 6
- **(b)** Index of min and max
- **(c)** Second-largest value (no `sort`)
- **(d)** Local maxima: elements greater than both immediate neighbours (ignore endpoints)

Sol 3

```r
x <- c(3, 7, 2, 9, 1, 5, 8, 4, 6, 10)
which(x > 6)                                    # (a)
```

```
# [1]  2  4  7 10
```

```r
which.min(x); which.max(x)                      # (b)
```

```
# [1] 5
```

```
# [1] 10
```

```r
max(x[x != max(x)])                             # (c)
```

```
# [1] 9
```

```r
n <- length(x); idx <- 2:(n-1)                  # (d)
x[idx][x[idx] > x[idx-1] & x[idx] > x[idx+1]]
```

```
# [1] 7 9 8
```

Ex 4: Cumulative and Parallel Functions

Problem

Let `x <- c(2,5,3,8,1,7,4)` and `y <- c(6,2,9,1,4,3,8)`.

- **(a)** Running maximum of `x`
- **(b)** Running sum and product
- **(c)** Element-wise min and max
- **(d)** Verify $\sum \text{pmin}(x, y) + \sum \text{pmax}(x, y) = \sum x + \sum y$

Sol 4

```r
x <- c(2,5,3,8,1,7,4); y <- c(6,2,9,1,4,3,8)
cummax(x)                                          # (a)
```

```
# [1] 2 5 5 8 8 8 8
```

```r
cumsum(x); cumprod(x)                              # (b)
```

```
# [1]  2  7 10 18 19 26 30

# [1]    2   10   30  240  240 1680 6720
```

```r
pmin(x,y); pmax(x,y)                              # (c)
```

```
# [1] 2 2 3 1 1 3 4

# [1] 6 5 9 8 4 7 8
```

```r
sum(pmin(x,y)) + sum(pmax(x,y)) == sum(x) + sum(y)# (d)
```

```
# [1] TRUE
```

Ex 5: Set Operations

Problem

Let `a <- c(1,3,5,7,9,3,5)` and `b <- c(2,3,5,6,7,2)`.

- **(a)** Unique elements of `a`
- **(b)** Positions of duplicated values in `a`
- **(c)** $a \cup b$, $a \cap b$, $a \setminus b$
- **(d)** `match(c(3,5,4), b)` — explain the `NA`

Sol 5

```r
a <- c(1,3,5,7,9,3,5); b <- c(2,3,5,6,7,2)
unique(a)                                          # (a)
```

```
# [1] 1 3 5 7 9
```

```r
which(duplicated(a))                               # (b)
```

```
# [1] 6 7
```

```r
union(a,b); intersect(a,b); setdiff(a,b)          # (c)
```

```
# [1] 1 3 5 7 9 2 6

# [1] 3 5 7

# [1] 1 9
```

```r
match(c(3,5,4), b)  # 4 not found -> NA            # (d)
```

```
# [1]  2  3 NA
```

Ex 6: Recycling Rule

Problem

- **(a)** What does `c(1,2,3,4,5,6) + c(10,20)` return? Why?
- **(b)** Let `m <- matrix(1:12, nrow=3)`. What does `m + c(100,200,300)` do?
- **(c)** Extract odd-indexed and even-indexed elements of `1:12` using `c(TRUE,FALSE)`
- **(d)** Generate `1 0 1 0 1 0 1 0 1 0` using `rep`

Sol 6

```r
c(1,2,3,4,5,6) + c(10,20)                          # (a) recycled
```

```
# [1] 11 22 13 24 15 26
```

```r
m <- matrix(1:12, nrow=3)
m + c(100,200,300)    # adds per column (col-major) # (b)
```

```
#      [,1] [,2] [,3] [,4]
# [1,]  101  104  107  110
# [2,]  202  205  208  211
# [3,]  303  306  309  312
```

```r
x <- 1:12; x[c(T,F)]; x[c(F,T)]                  # (c)
```

```
# [1]  1  3  5  7  9 11

# [1]  2  4  6  8 10 12
```

```r
rep(c(1,0), 5)                                     # (d)
```

```
#  [1] 1 0 1 0 1 0 1 0 1 0
```

Ex 7: Sorting and Ranking

Problem

Let `x <- c(40,15,30,10,50,25)`.

- **(a)** Sort ascending and descending
- **(b)** Find `order(x)` and verify `x[order(x)] == sort(x)`
- **(c)** What rank does 30 receive?
- **(d)** Sort `c("banana","apple","cherry")` alphabetically

Sol 7

```r
x <- c(40,15,30,10,50,25)
sort(x); sort(x, decreasing=TRUE)                    # (a)
```

```
# [1] 10 15 25 30 40 50

# [1] 50 40 30 25 15 10
```

```r
ord <- order(x); all(x[ord] == sort(x))              # (b)
```

```
# [1] TRUE
```

```r
rank(x)  # 30 -> rank 4                              # (c)
```

```
# [1] 5 2 4 1 6 3
```

```r
sort(c("banana","apple","cherry"))                    # (d)
```

```
# [1] "apple"  "banana" "cherry"
```

Ex 8: NA, NULL, NaN, Inf

Problem

- **(a)** Given `x <- c(3, NA, -1, NaN, Inf, -Inf, 0)`, apply `is.na`, `is.nan`, `is.finite`, `is.infinite`. Why is `is.na(NaN)` TRUE?
- **(b)** Remove non-finite values and compute mean
- **(c)** Show `c(1, NULL, 3)` vs `c(1, NA, 3)` lengths
- **(d)** Compute `NA\textasciicircum{0}`, `NA | TRUE`, `NA \& FALSE`

Sol 8

```r
x <- c(3, NA, -1, NaN, Inf, -Inf, 0)
is.na(x); is.nan(x); is.finite(x)                    # (a)
```

```
# [1] FALSE  TRUE FALSE  TRUE FALSE FALSE FALSE

# [1] FALSE FALSE FALSE  TRUE FALSE FALSE FALSE

# [1]  TRUE FALSE  TRUE FALSE FALSE FALSE  TRUE
```

```r
mean(x[is.finite(x)])                                 # (b)
```

```
# [1] 0.6666667
```

```r
length(c(1, NULL, 3)); length(c(1, NA, 3))            # (c) 2 vs 3
```

```
# [1] 2

# [1] 3
```

```r
NA^0; NA | TRUE; NA & FALSE                           # (d)
```

```
# [1] 1

# [1] TRUE

# [1] FALSE
```

Ex 9: Type Coercion

Problem

- **(a)** What does `c(TRUE, 1L, 2.5, "x")` produce? What type?
- **(b)** Coerce `c("3","1.5","NA","7")` to numeric. What happens to `"NA"`?
- **(c)** Predict `as.integer(c(3.9, -2.7, TRUE, "5"))`
- **(d)** Why does `1 == "1"` return `TRUE`?

Sol 9

```r
x <- c(TRUE, 1L, 2.5, "x"); x; typeof(x)          # (a) character
```

```
# [1] "TRUE" "1"    "2.5"  "x"

# [1] "character"
```

```r
as.numeric(c("3","1.5","NA","7"))                  # (b) NA + warning
```

```
# [1] 3.0 1.5  NA 7.0
```

```r
as.integer(c(3.9, -2.7, TRUE, "5"))               # (c) 3,-2,1,5
```

```
# [1]  3 -2 NA  5
```

```r
1 == "1"  # 1 coerced to "1", then string compare  # (d)
```

```
# [1] TRUE
```

Ex 10: `seq` and `seq_along`

Problem

- **(a)** Generate `1.0 2.5 4.0 5.5 7.0 8.5 10.0` using `seq`
- **(b)** Show `1:length(x)` is unsafe when `x <- NULL`; fix with `seq\_along`
- **(c)** 6 equally-spaced values from 0 to 2π
- **(d)** What does `seq\_along(integer(0))` return?

Sol 10

```r
seq(1, 10, by=1.5)                                  # (a)
```

```
# [1]  1.0  2.5  4.0  5.5  7.0  8.5 10.0
```

```r
x <- NULL; 1:length(x)                              # (b) c(1,0) WRONG
```

```
# [1] 1 0
```

```r
seq_along(x)                                         # integer(0) SAFE
```

```
# integer(0)
```

```r
seq(0, 2*pi, length.out=6)                          # (c)
```

```
# [1] 0.000000 1.256637 2.513274 3.769911 5.026548 6.283185
```

```r
seq_along(integer(0))                                # (d) integer(0)
```

```
# integer(0)
```

Ex 11: `rep` Patterns

Problem

Using `rep`, generate:

- **(a)** `1 1 2 2 3 3 4 4 5 5`
- **(b)** `1 2 3 1 2 3 1 2 3 1 2 3`
- **(c)** `"A" "A" "A" "B" "B" "C"`
- **(d)** Alternating `TRUE FALSE` of length 20

Sol 11

```r
rep(1:5, each=2)                                    # (a)
```

```
#  [1] 1 1 2 2 3 3 4 4 5 5
```

```r
rep(1:3, times=4)                                   # (b)
```

```
#  [1] 1 2 3 1 2 3 1 2 3 1 2 3
```

```r
rep(c("A","B","C"), c(3,2,1))                      # (c)
```

```
# [1] "A" "A" "A" "B" "B" "C"
```

```r
rep(c(TRUE,FALSE), 10)                              # (d)
```

```
#  [1]  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE
# [13]  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE
```

Ex 12: `%in%` and `match`

Problem

`haystack <- c(3,1,4,1,5,9,2,6,5,3)`; `needles <- c(5,7,2)`.

- **(a)** Which needles are present?
- **(b)** First occurrence position of each
- **(c)** Difference between `match("a", c("b","a","a"))` and `which(c("b","a","a") == "a")`
- **(d)** Filter `mtcars` rows where `cyl %in% c(4,6)`

Sol 12

```r
haystack <- c(3,1,4,1,5,9,2,6,5,3); needles <- c(5,7,2)
needles %in% haystack                                  # (a)
```

```
# [1]  TRUE FALSE  TRUE
```

```r
match(needles, haystack)                               # (b) NA for 7
```

```
# [1]  5 NA  7
```

```r
match("a", c("b","a","a"))                             # (c) 2 (first)
```

```
# [1] 2
```

```r
which(c("b","a","a") == "a")                           # c(2,3) (all)
```

```
# [1] 2 3
```

```r
nrow(mtcars[mtcars$cyl %in% c(4,6), ])                # (d)
```

```
# [1] 18
```

Ex 13: Vector Arithmetic

Problem

- **(a)** Compute $H_{100} = \sum_{k=1}^{100} \frac{1}{k}$ in one line
- **(b)** Compute $\sum_{k=0}^{50} \frac{(-1)^k}{k!}$. Compare with `exp(-1)`
- **(c)** For `x <- seq(0.1,2,by=0.1)`, compute $e^x \sin x - \ln x$, rounded to 4 dp
- **(d)** Compute `diff(cumsum(1:10))` and explain

Sol 13

```r
sum(1 / 1:100)                                    # (a) H_100
```

```
# [1] 5.187378
```

```r
k <- 0:50; sum((-1)^k / factorial(k)); exp(-1)    # (b)
```

```
# [1] 0.3678794
```

```
# [1] 0.3678794
```

```r
x <- seq(0.1, 2, by=0.1)
round(exp(x)*sin(x) - log(x), 4)                  # (c)
```

```
#  [1] 2.4129 1.8521 1.6029 1.4972 1.4836 1.5397 1.6540 1.8196 2.0320 2.2874
# [11] 2.5820 2.9122 3.2732 3.6597 4.0650 4.4809 4.8977 5.3036 5.6850 6.0257
```

```r
diff(cumsum(1:10))  # recovers 2:10                # (d)
```

```
# [1]  2  3  4  5  6  7  8  9 10
```

Ex 14: Attributes and `structure`

Problem

- **(a)** Create `a <- 1:4`; attach attributes `"unit"="kg"` and `"source"="lab"`. Print
- **(b)** Recreate using `structure()` in one call
- **(c)** `x <- 1:3` vs `y <- c(1,2,3)`: are they `identical`? `all.equal`? Why?
- **(d)** When to use `isTRUE(all.equal())` in an `if`?

Sol 14

```r
a <- 1:4; attr(a,"unit") <- "kg"; attr(a,"source") <- "lab"
attributes(a)                                          # (a)
```

```
# $unit
# [1] "kg"
#
# $source
# [1] "lab"
```

```r
b <- structure(1:4, unit="kg", source="lab")           # (b)
x <- 1:3; y <- c(1,2,3)
identical(x, y); all.equal(x, y)                       # (c) F; T
```

```
# [1] FALSE
```

```
# [1] TRUE
```

```r
isTRUE(all.equal(0.1+0.2, 0.3))                       # (d) TRUE
```

```
# [1] TRUE
```

Ex 15: Factors

Problem

- **(a)** Create factor `f <- factor(c("low","med","high","med","low"))`. What are the levels?
- **(b)** Create an ordered factor with levels `low < med < high`. Test `f > "low"`
- **(c)** Use `table(f)` to count occurrences
- **(d)** What does `as.integer(f)` return and why?

Sol 15

```r
f <- factor(c("low","med","high","med","low"))
levels(f)                                          # (a) alphabetical
```

```
# [1] "high" "low"  "med"
```

```r
fo <- factor(f, levels=c("low","med","high"), ordered=TRUE)
fo > "low"                                         # (b)
```

```
# [1] FALSE  TRUE  TRUE  TRUE FALSE
```

```r
table(f)                                           # (c)
```

```
# f
# high  low  med
#    1    2    2
```

```r
as.integer(f)  # underlying integer codes          # (d)
```

```
# [1] 2 3 1 3 2
```

Ex 16: Floating Point

Problem

- **(a)** Is `0.1 + 0.2 == 0.3` TRUE or FALSE? Why?
- **(b)** Use `all.equal` to compare correctly
- **(c)** What is `trunc(-3.7)` vs `floor(-3.7)`?
- **(d)** Demonstrate banker's rounding: `round(0.5)`, `round(1.5)`, `round(2.5)`

Sol 16

```r
0.1 + 0.2 == 0.3  # FALSE (floating point)        # (a)
```

```
# [1] FALSE
```

```r
all.equal(0.1 + 0.2, 0.3)                         # (b) TRUE
```

```
# [1] TRUE
```

```r
trunc(-3.7); floor(-3.7)  # -3 vs -4              # (c)
```

```
# [1] -3

# [1] -4
```

```r
round(0.5); round(1.5); round(2.5)  # 0; 2; 2    # (d)
```

```
# [1] 0

# [1] 2

# [1] 2
```

Ex 17: Appending and Inserting

Problem

- **(a)** Start with `v <- 1:5`. Append `c(10,11)` to the end
- **(b)** Insert 99 after position 3 using `append`
- **(c)** What happens if you assign `v[10] <- 42` when `length(v)` is 7?
- **(d)** Remove the 3rd element without using negative indexing (use logical)

Sol 17

```r
v <- 1:5; v <- c(v, c(10,11)); v                    # (a)
```

```
# [1]  1  2  3  4  5 10 11
```

```r
append(v, 99, after=3)                               # (b)
```

```
# [1]  1  2  3 99  4  5 10 11
```

```r
v[10] <- 42; v   # NAs fill gaps                     # (c)
```

```
#  [1]  1  2  3  4  5 10 11 NA NA 42
```

```r
v[seq_along(v) != 3]                                 # (d)
```

```
# [1]  1  2  4  5 10 11 NA NA 42
```

Ex 18: `sample` and `sequence`

Problem

- **(a)** Simulate 20 dice rolls with `sample`
- **(b)** Use `sample` to shuffle `letters[1:10]`
- **(c)** Using `sequence`, generate `1 2 3 4  1 2 3  1 2  1`
- **(d)** Generate `5 4 3 2 1  6 5 4 3 2 1` using `sequence`

Sol 18

```r
set.seed(42)
sample(1:6, 20, replace=TRUE)                    # (a)
```

```
#  [1] 1 5 1 1 2 4 2 2 1 4 1 5 6 4 2 2 3 1 1 3
```

```r
sample(letters[1:10])                             # (b)
```

```
#  [1] "d" "e" "g" "i" "h" "j" "b" "c" "f" "a"
```

```r
sequence(4:1)                                     # (c)
```

```
#  [1] 1 2 3 4 1 2 3 1 2 1
```

```r
sequence(5:6, from=5:6, by=-1)                    # (d)
```

```
#  [1] 5 4 3 2 1 6 5 4 3 2 1
```

Operators and Common Functions (Ex 19–26)

Ex 19: Operator Precedence

Problem

- **(a)** Without running, predict: `2 + 3 * 4`, `2\textasciicircum{3^2}`, `1:3 + 1`, `1:(3+1)`
- **(b)** Why does `-2\textasciicircum{2}` give `-4` instead of 4?
- **(c)** Predict: `TRUE | FALSE \& FALSE`
- **(d)** Write `x <- 5; x %% 3 == 2 \& x > 3` and trace operator precedence

Sol 19

```r
2 + 3 * 4     # 14: * before +                    # (a)
```

```
# [1] 14
```

```r
2^3^2          # 512: ^ right-associative = 2^9
```

```
# [1] 512
```

```r
1:3 + 1        # 2 3 4: : before +
```

```
# [1] 2 3 4
```

```r
1:(3+1)        # 1 2 3 4
```

```
# [1] 1 2 3 4
```

```r
-2^2           # -4: ^ before unary -              # (b)
```

```
# [1] -4
```

```r
TRUE | FALSE & FALSE  # TRUE: & before |           # (c)
```

```
# [1] TRUE
```

```r
x <- 5; x %% 3 == 2 & x > 3  # TRUE               # (d)
```

```
# [1] TRUE
```

Ex 20: Math Functions

Problem

- **(a)** Compute `floor`, `ceiling`, `trunc`, `round` of `-5.7`
- **(b)** Compute $\binom{10}{3}$ and verify with `factorial`
- **(c)** Compute `119 %/% 13` and `119 %% 13`. Verify `119 == 13 * (119 %/% 13) + 119 %% 13`
- **(d)** `signif(12345678, 4)`

Sol 20

```r
floor(-5.7); ceiling(-5.7); trunc(-5.7); round(-5.7) # (a)
```

```
# [1] -6

# [1] -5

# [1] -5

# [1] -6
```

```r
choose(10, 3); factorial(10)/(factorial(3)*factorial(7)) # (b)
```

```
# [1] 120

# [1] 120
```

```r
119 %/% 13; 119 %% 13                                    # (c)
```

```
# [1] 9

# [1] 2
```

```r
119 == 13 * (119 %/% 13) + 119 %% 13                     # TRUE
```

```
# [1] TRUE
```

```r
signif(12345678, 4)                                       # (d)
```

Ex 21: Statistical Summary

Problem

`v <- c(4, 7, 2, 9, 1, 5, 8, 3, 6, 10)`.

- **(a)** mean, median, var, sd, range
- **(b)** `quantile(v, c(0.25, 0.75))` and IQR
- **(c)** Compute z-scores: $(v - \bar{v})/s$. How many exceed $|z| > 1$?
- **(d)** Use `diff(v)` to find the largest jump

Sol 21

```r
v <- c(4, 7, 2, 9, 1, 5, 8, 3, 6, 10)
mean(v); median(v); var(v); sd(v); range(v)        # (a)
```

```
# [1] 5.5

# [1] 5.5

# [1] 9.166667

# [1] 3.02765

# [1]  1 10
```

```r
quantile(v, c(0.25, 0.75)); IQR(v)                # (b)
```

```
#  25%  75%
# 3.25 7.75

# [1] 4.5
```

```r
z <- (v - mean(v)) / sd(v); sum(abs(z) > 1)       # (c)
```

```
# [1] 4
```

```r
max(abs(diff(v)))                                  # (d) largest jump
```

```
# [1] 8
```

Ex 22: `table` and `prop.table`

Problem

`x <- c("A","B","A","C","B","A","C","C","B","A")`.

- **(a)** Frequency table of `x`
- **(b)** Proportion table
- **(c)** Create `y <- c(1,1,2,1,2,2,1,2,1,2)`. Cross-tabulate `x` and `y`; add margins
- **(d)** Which category of `x` is most frequent?

Sol 22

```r
x <- c("A","B","A","C","B","A","C","C","B","A")
table(x)                                                    # (a)
```

```
# x
# A B C
# 4 3 3
```

```r
prop.table(table(x))                                        # (b)
```

```
# x
#   A   B   C
# 0.4 0.3 0.3
```

```r
y <- c(1,1,2,1,2,2,1,2,1,2)
addmargins(table(x, y))                                     # (c)
```

```
#       y
# x       1  2 Sum
#   A     1  3   4
#   B     2  1   3
#   C     2  1   3
#   Sum   5  5  10
```

```r
names(which.max(table(x)))                                  # (d) "A"
```

```
# [1] "A"
```

Ex 23: Harmonic Series Convergence

Problem

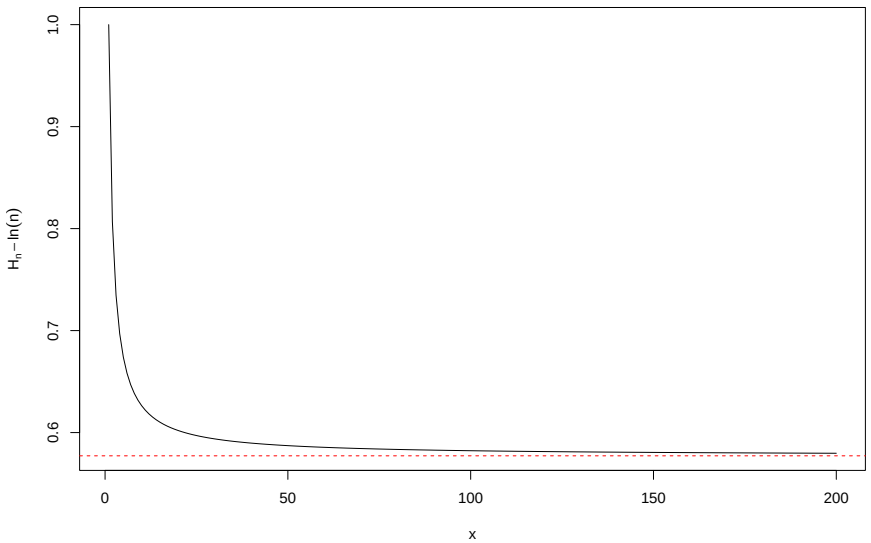
- **(a)** Compute $H_n = \sum_{k=1}^{n} \frac{1}{k}$ for $n \in \{10, 100, 1000, 10000\}$
- **(b)** Compare each H_n with $\ln(n) + \gamma$ where $\gamma \approx 0.5772$
- **(c)** Plot $H_n - \ln n$ for $n = 1, \ldots, 200$

Sol 23

```r
ns <- c(10, 100, 1000, 10000)                          # (a)
Hn <- sapply(ns, function(n) sum(1/1:n))
data.frame(n=ns, Hn=round(Hn,4),
           approx=round(log(ns)+0.5772,4))             # (b)
```

```
#       n     Hn approx
# 1    10 2.9290 2.8798
# 2   100 5.1874 5.1824
# 3  1000 7.4855 7.4850
# 4 10000 9.7876 9.7875
```

```r
x <- 1:200; y <- cumsum(1/x) - log(x)                  # (c)
plot(x, y, type="l", ylab=expression(H[n]-ln(n)))
abline(h=0.5772, col="red", lty=2)
```



Ex 24: `pmin`/`pmax` Application

Problem

- **(a)** Clip vector `v <- c(-5,3,12,-2,8,25)` to range $[0, 10]$ using `pmin` and `pmax`
- **(b)** Given two portfolios `a <- c(5,8,3,9)` and `b <- c(7,2,6,4)`, compute the "best of both" portfolio
- **(c)** Compute $\sum_{k=1}^{100} \min(2^k, k^4)$ without loops

Sol 24

```r
v <- c(-5, 3, 12, -2, 8, 25)
pmin(pmax(v, 0), 10)                                  # (a) clip to [0,10]
```

```
# [1]  0  3 10  0  8 10
```

```r
a <- c(5,8,3,9); b <- c(7,2,6,4)
pmax(a, b)                                            # (b)
```

```
# [1] 7 8 6 9
```

```r
k <- 1:100; sum(pmin(2^k, k^4))                      # (c)
```

```
# [1] 2050220551
```

Ex 25: Cumulative Functions

Problem

`x <- c(3, -1, 4, -1, 5, -9, 2, 6)`.

- **(a)** Compute `cumsum(x)`. At which index does it first become negative?
- **(b)** Maximum drawdown: $\max_i(\text{cummax}_i - \text{cumsum}_i)$
- **(c)** Compute `cumprod(1 + c(0.05, -0.02, 0.03, -0.01))` (portfolio growth)

Sol 25

```r
x <- c(3, -1, 4, -1, 5, -9, 2, 6)
cs <- cumsum(x); cs                                  # (a)
```

```
# [1]  3  2  6  5 10  1  3  9
```

```r
which(cs < 0)[1]                                     # first negative
```

```
# [1] NA
```

```r
cm <- cummax(cs); max(cm - cs)                       # (b) max drawdown
```

```
# [1] 9
```

```r
cumprod(1 + c(0.05, -0.02, 0.03, -0.01))            # (c) growth
```

```
# [1] 1.050000 1.029000 1.059870 1.049271
```

Ex 26: `order` and Data Sorting

Problem

```
d <- data.frame(name=c("Zara","Amy","Mike","Bob"), score=c(88,95,72,95),
age=c(22,20,25,21)).
```

- **(a)** Sort by score ascending
- **(b)** Sort by score descending, ties broken by age ascending
- **(c)** Find the name of the person with the highest score (youngest if tied)

Sol 26

```r
d <- data.frame(name=c("Zara","Amy","Mike","Bob"),
                score=c(88,95,72,95), age=c(22,20,25,21))
d[order(d$score), ]                                  # (a)
```

```
#   name score age
# 3 Mike    72  25
# 1 Zara    88  22
# 2  Amy    95  20
# 4  Bob    95  21
```

```r
d[order(-d$score, d$age), ]                          # (b)
```

```
#   name score age
# 2  Amy    95  20
# 4  Bob    95  21
# 1 Zara    88  22
# 3 Mike    72  25
```

```r
d[order(-d$score, d$age), ]$name[1]                  # (c) Bob
```

```
# [1] "Amy"
```

Data Frames (Ex 27–40)

Ex 27: Creating and Inspecting

Problem

- **(a)** Create a data frame with columns `name` (Alice, Bob, Cathy), `age` (25, 30, 28), `score` (88.5, 92.3, 76.8)
- **(b)** Show `str`, `dim`, `names`, `summary`
- **(c)** What class is `df\$name`?
- **(d)** Access `score` of row 2 by name and by position

Sol 27

```r
df <- data.frame(name=c("Alice","Bob","Cathy"),
                 age=c(25,30,28), score=c(88.5,92.3,76.8))
str(df); dim(df); names(df)                          # (b)
```

```
# 'data.frame': 3 obs. of  3 variables:
#  $ name : chr  "Alice" "Bob" "Cathy"
#  $ age  : num  25 30 28
#  $ score: num  88.5 92.3 76.8

# [1] 3 3

# [1] "name"  "age"   "score"
```

```r
class(df$name)                                       # (c)
```

```
# [1] "character"
```

```r
df$score[2]; df[2, "score"]                          # (d)
```

```
# [1] 92.3

# [1] 92.3
```

Ex 28: Row Filtering

Problem

Using `tg <- ToothGrowth`:

- **(a)** Rows where `supp == "OJ"` and `dose >= 1`
- **(b)** Mean tooth length for VC vs OJ
- **(c)** Rows where `len` is above its overall median
- **(d)** Same as (a) but using `subset()`

Sol 28

```r
tg <- ToothGrowth
head(tg[tg$supp == "OJ" & tg$dose >= 1, ], 4)     # (a)
```

```
#     len supp dose
# 41 19.7   OJ    1
# 42 23.3   OJ    1
# 43 23.6   OJ    1
# 44 26.4   OJ    1
```

```r
tapply(tg$len, tg$supp, mean)                     # (b)
```

```
#       OJ       VC
# 20.66333 16.96333
```

```r
head(tg[tg$len > median(tg$len), ], 4)            # (c)
```

```
#     len supp dose
# 15 22.5   VC    1
# 21 23.6   VC    2
# 23 33.9   VC    2
# 24 25.5   VC    2
```

```r
head(subset(tg, supp=="OJ" & dose >= 1), 4)       # (d)
```

```
#     len supp dose
# 41 19.7   OJ    1
# 42 23.3   OJ    1
```

Ex 29: Adding and Removing Columns

Problem

Start with `df <- data.frame(x=1:5, y=c(10,20,30,40,50))`.

- **(a)** Add column `z = x * y`
- **(b)** Add column `label` using `paste0("item", 1:5)`
- **(c)** Remove column `z`
- **(d)** Rename `y` to `value`

Sol 29

```r
df <- data.frame(x=1:5, y=c(10,20,30,40,50))
df$z <- df$x * df$y                                  # (a)
df$label <- paste0("item", 1:5)                       # (b)
df$z <- NULL                                          # (c)
names(df)[names(df)=="y"] <- "value"; df              # (d)
```

```
#   x value label
# 1 1    10 item1
# 2 2    20 item2
# 3 3    30 item3
# 4 4    40 item4
# 5 5    50 item5
```

Ex 30: `with()` and `within()`

Problem

Using `tg <- ToothGrowth`:

- **(a)** Use `with()` to compute `mean(len[dose == 2])`
- **(b)** Use `within()` to add `log\_len <- log(len)` and `len\_centered <- len - mean(len)`

Sol 30

```r
tg <- ToothGrowth
with(tg, mean(len[dose == 2]))                          # (a)
```

```
# [1] 26.1
```

```r
tg2 <- within(tg, {
  log_len <- log(len)
  len_centered <- len - mean(len)
})
head(tg2, 3)                                            # (b)
```

```
#    len supp dose len_centered  log_len
# 1  4.2   VC  0.5   -14.613333 1.435085
# 2 11.5   VC  0.5    -7.313333 2.442347
# 3  7.3   VC  0.5   -11.513333 1.987874
```

Ex 31: Handling NAs

Problem

`df <- data.frame(x=c(1,NA,3,4,NA), y=c(NA,2,3,NA,5), z=c(1,2,3,4,5)).`

- **(a)** Which rows are complete?
- **(b)** Remove all incomplete rows
- **(c)** Replace NAs in column `x` with the mean of `x` (ignoring NAs)
- **(d)** Count NAs per column

Sol 31

```r
df <- data.frame(x=c(1,NA,3,4,NA), y=c(NA,2,3,NA,5), z=1:5)
complete.cases(df)                                    # (a)
```

```
# [1] FALSE FALSE  TRUE FALSE FALSE
```

```r
df[complete.cases(df), ]                              # (b)
```

```
#   x y z
# 3 3 3 3
```

```r
df$x[is.na(df$x)] <- mean(df$x, na.rm=TRUE); df   # (c)
```

```
#          x  y z
# 1 1.000000 NA 1
# 2 2.666667  2 2
# 3 3.000000  3 3
# 4 4.000000 NA 4
# 5 2.666667  5 5
```

```r
colSums(is.na(df))                                    # (d)
```

```
# x y z
# 0 2 0
```

Ex 32: `merge()`

Problem

```
students <- data.frame(id=1:4, name=c("A","B","C","D")). grades <-
data.frame(sid=c(1,2,2,4), course=c("Math","Math","Eng","Eng"),
grade=c(85,90,78,92)).
```

- **(a)** Inner join on id
- **(b)** Left join (keep all students)
- **(c)** How many rows in each result? Why?

Sol 32

```r
students <- data.frame(id=1:4, name=c("A","B","C","D"))
grades <- data.frame(sid=c(1,2,2,4),
  course=c("Math","Math","Eng","Eng"), grade=c(85,90,78,92))
merge(students, grades, by.x="id", by.y="sid")     # (a) inner
```

```
#   id name course grade
# 1  1    A   Math    85
# 2  2    B   Math    90
# 3  2    B    Eng    78
# 4  4    D    Eng    92
```

```r
merge(students, grades, by.x="id", by.y="sid",
      all.x=TRUE)                                  # (b) left
```

```
#   id name course grade
# 1  1    A   Math    85
# 2  2    B   Math    90
# 3  2    B    Eng    78
# 4  3    C   <NA>    NA
# 5  4    D    Eng    92
```

```r
# inner: 4 rows; left: 5 rows (C has NA)           # (c)
```

Ex 33: `aggregate()`

Problem

Using `mtcars`:

- **(a)** Mean `mpg` by `cyl`
- **(b)** Mean and sd of `mpg` by `cyl` and `am`
- **(c)** Max `hp` by `cyl`

Sol 33

```r
aggregate(mpg ~ cyl, data=mtcars, FUN=mean)        # (a)
```

```
#   cyl      mpg
# 1   4 26.66364
# 2   6 19.74286
# 3   8 15.10000
```

```r
aggregate(mpg ~ cyl + am, data=mtcars,
  FUN=function(x) c(m=mean(x), s=sd(x)))           # (b)
```

```
#   cyl am      mpg.m      mpg.s
# 1   4  0 22.9000000  1.4525839
# 2   6  0 19.1250000  1.6317169
# 3   8  0 15.0500000  2.7743959
# 4   4  1 28.0750000  4.4838599
# 5   6  1 20.5666667  0.7505553
# 6   8  1 15.4000000  0.5656854
```

```r
aggregate(hp ~ cyl, data=mtcars, FUN=max)          # (c)
```

```
#   cyl  hp
# 1   4 113
# 2   6 175
# 3   8 335
```

Ex 34: `cbind` and `rbind`

Problem

- **(a)** Create two 1-column data frames and `cbind` them
- **(b)** Create two data frames with same columns and `rbind`
- **(c)** Use `do.call(rbind, list\_of\_dfs)` with 3 data frames

Sol 34

```r
d1 <- data.frame(a=1:3); d2 <- data.frame(b=4:6)
cbind(d1, d2)                                          # (a)
```

```
#   a b
# 1 1 4
# 2 2 5
# 3 3 6
```

```r
e1 <- data.frame(x="a", y=1); e2 <- data.frame(x="b", y=2)
rbind(e1, e2)                                          # (b)
```

```
#   x y
# 1 a 1
# 2 b 2
```

```r
dfs <- list(data.frame(x=1,y=2), data.frame(x=3,y=4),
            data.frame(x=5,y=6))
do.call(rbind, dfs)                                    # (c)
```

```
#   x y
# 1 1 2
# 2 3 4
# 3 5 6
```

Ex 35: Data Frame as Named List

Problem

- **(a)** Show `is.list(mtcars)` is TRUE. What does `typeof(mtcars)` return?
- **(b)** Access the `mpg` column using `[[` notation
- **(c)** What is the difference between `mtcars["mpg"]` and `mtcars[["mpg"]]`?
- **(d)** Convert `mtcars[1:3, 1:3]` to a list and back

Sol 35

```r
is.list(mtcars); typeof(mtcars)                    # (a) TRUE; "list"
```

```
# [1] TRUE

# [1] "list"
```

```r
head(mtcars[["mpg"]])                              # (b)
```

```
# [1] 21.0 21.0 22.8 21.4 18.7 18.1
```

```r
class(mtcars["mpg"]); class(mtcars[["mpg"]])       # (c) df vs numeric
```

```
# [1] "data.frame"

# [1] "numeric"
```

```r
l <- as.list(mtcars[1:3, 1:3]); as.data.frame(l)  # (d)
```

```
#    mpg cyl disp
# 1 21.0   6  160
# 2 21.0   6  160
# 3 22.8   4  108
```

Ex 36: `split()` on Data Frames

Problem

- **(a)** Split `iris` by `Species`. What type is the result?
- **(b)** Compute mean `Sepal.Length` per species using `split` + `sapply`
- **(c)** Recombine with `do.call(rbind, ...)`

Sol 36

```r
sp <- split(iris, iris$Species)
class(sp); names(sp)                                  # (a) list
```

```
# [1] "list"

# [1] "setosa"     "versicolor" "virginica"
```

```r
sapply(sp, function(d) mean(d$Sepal.Length))          # (b)
```

```
#     setosa versicolor  virginica
#      5.006      5.936      6.588
```

```r
head(do.call(rbind, sp), 4)                           # (c)
```

```
#          Sepal.Length Sepal.Width Petal.Length Petal.Width Species
# setosa.1          5.1         3.5          1.4         0.2  setosa
# setosa.2          4.9         3.0          1.4         0.2  setosa
# setosa.3          4.7         3.2          1.3         0.2  setosa
# setosa.4          4.6         3.1          1.5         0.2  setosa
```

Ex 37: `subset()` with `select`

Problem

Using `mtcars`:

- **(a)** Select only `mpg`, `cyl`, `hp` for cars with `mpg > 25`
- **(b)** Exclude `drat` and `wt` columns
- **(c)** Why can you write `subset(mtcars, mpg > 25)` without quoting `mpg`?

Sol 37

```r
subset(mtcars, mpg > 25, select=c(mpg, cyl, hp))  # (a)
```

```
#                  mpg cyl  hp
# Fiat 128        32.4   4  66
# Honda Civic     30.4   4  52
# Toyota Corolla 33.9   4  65
# Fiat X1-9       27.3   4  66
# Porsche 914-2  26.0   4  91
# Lotus Europa    30.4   4 113
```

```r
head(subset(mtcars, select=-c(drat, wt)), 3)       # (b)
```

```
#                 mpg cyl disp  hp  qsec vs am gear carb
# Mazda RX4      21.0   6  160 110 16.46  0  1    4    4
# Mazda RX4 Wag 21.0   6  160 110 17.02  0  1    4    4
# Datsun 710     22.8   4  108  93 18.61  1  1    4    1
```

```r
# (c) subset uses non-standard evaluation: it captures
# the expression and evaluates it within the data frame
```

Ex 38: Sorting with Multiple Keys

Problem

```
df <- data.frame(dept=c("A","B","A","B","A"),
name=c("Zoe","Amy","Bob","Cal","Amy"), salary=c(50,60,70,55,65)).
```

- **(a)** Sort by dept ascending, then salary descending
- **(b)** Find the highest-paid person in each department (using `order` + `!duplicated`)

Sol 38

```r
df <- data.frame(dept=c("A","B","A","B","A"),
  name=c("Zoe","Amy","Bob","Cal","Amy"), salary=c(50,60,70,55,65))
df[order(df$dept, -df$salary), ]                    # (a)
```

```
#   dept name salary
# 3    A  Bob     70
# 5    A  Amy     65
# 1    A  Zoe     50
# 2    B  Amy     60
# 4    B  Cal     55
```

```r
sorted <- df[order(df$dept, -df$salary), ]
sorted[!duplicated(sorted$dept), ]                  # (b)
```

```
#   dept name salary
# 3    A  Bob     70
# 2    B  Amy     60
```

Ex 39: Column-wise Operations

Problem

- **(a)** Use `colMeans` on `mtcars[,1:4]`
- **(b)** Use `sapply` to find `max` of each column
- **(c)** Standardize (z-score) all numeric columns using `scale()`. Compare with manual `sweep`

Sol 39

```r
colMeans(mtcars[,1:4])                                  # (a)
```

```
#       mpg       cyl      disp        hp
#  20.09062   6.18750 230.72188 146.68750
```

```r
sapply(mtcars[,1:4], max)                               # (b)
```

```
#   mpg   cyl  disp    hp
#  33.9   8.0 472.0 335.0
```

```r
head(scale(mtcars[,1:2]), 3)                            # (c)
```

```
#                     mpg        cyl
# Mazda RX4     0.1508848 -0.1049878
# Mazda RX4 Wag 0.1508848 -0.1049878
# Datsun 710    0.4495434 -1.2248578
```

```r
m <- as.matrix(mtcars[,1:2])
head(sweep(sweep(m, 2, colMeans(m)), 2, apply(m,2,sd), "/"), 3)
```

```
#                     mpg        cyl
# Mazda RX4     0.1508848 -0.1049878
# Mazda RX4 Wag 0.1508848 -0.1049878
# Datsun 710    0.4495434 -1.2248578
```

Ex 40: Building a Gradebook

Problem

- **(a)** Create `grades <- data.frame(name=c("A","B","C","D","E"), hw=c(85,92,78,95,88), exam=c(90,85,92,88,76))`
- **(b)** Add `final = 0.4*hw + 0.6*exam`
- **(c)** Add `letter` using nested `ifelse` (A/B/C/F)
- **(d)** Sort by `final` descending

Sol 40

```r
grades <- data.frame(name=c("A","B","C","D","E"),
  hw=c(85,92,78,95,88), exam=c(90,85,92,88,76))
grades$final <- 0.4*grades$hw + 0.6*grades$exam   # (b)
grades$letter <- ifelse(grades$final>=90,"A",      # (c)
  ifelse(grades$final>=80,"B",
  ifelse(grades$final>=70,"C","F")))
grades[order(-grades$final), ]                     # (d)
```

```
#   name hw exam final letter
# 4    D 95   88  90.8      A
# 1    A 85   90  88.0      B
# 2    B 92   85  87.8      B
# 3    C 78   92  86.4      B
# 5    E 88   76  80.8      B
```

Lists (Ex 41–52)

Ex 41: Creating and Accessing Lists

Problem

- **(a)** Create `l <- list(name="Alice", scores=c(85,92,78), passed=TRUE)`
- **(b)** Access `scores` with `\$`, `[[]]`, and `[]`. Show the different classes
- **(c)** Add element `grade = "B"`. Remove `passed`
- **(d)** What is `length(l)` before and after?

Sol 41

```r
l <- list(name="Alice", scores=c(85,92,78), passed=TRUE)
l$scores; l[["scores"]]; l["scores"]                # (b)
```

```
# [1] 85 92 78

# [1] 85 92 78

# $scores
# [1] 85 92 78
```

```r
class(l$scores); class(l["scores"])                 # vector vs list
```

```
# [1] "numeric"

# [1] "list"
```

```r
l$grade <- "B"; l$passed <- NULL                    # (c)
length(l)  # now 3                                  # (d)
```

```
# [1] 3
```

Ex 42: `unlist()` Behavior

Problem

- **(a)** `l <- list(a=1:3, b=4:5)`. What does `unlist(l)` produce? Explain the names
- **(b)** `unlist(l, use.names=FALSE)`
- **(c)** `l2 <- list(x="hello", y=1:3)`. What type is `unlist(l2)`? Why?
- **(d)** Mean of a list of numbers: `list(10, 20, 30)`

Sol 42

```r
l <- list(a=1:3, b=4:5)
unlist(l)                # named: a1 a2 a3 b1 b2   # (a)
```

```
# a1 a2 a3 b1 b2
#  1  2  3  4  5
```

```r
unlist(l, use.names=FALSE)                          # (b)
```

```
# [1] 1 2 3 4 5
```

```r
unlist(list(x="hello", y=1:3))  # all character    # (c)
```

```
#       x      y1      y2      y3
# "hello"     "1"     "2"     "3"
```

```r
mean(unlist(list(10, 20, 30)))                      # (d) 20
```

```
# [1] 20
```

Ex 43: Nested Lists

Problem

- **(a)** Create `student <- list(info=list(name="Bob", age=20), grades=list(math=90, eng=85))`
- **(b)** Access Bob's math grade
- **(c)** Add `grades\$sci = 92`
- **(d)** Use `str()` to visualize the structure

Sol 43

```r
student <- list(info=list(name="Bob", age=20),
                grades=list(math=90, eng=85))
student$grades$math                                  # (b) 90
```

```
# [1] 90
```

```r
student$grades$sci <- 92                             # (c)
str(student)                                         # (d)
```

```
# List of 2
#  $ info  :List of 2
#   ..$ name: chr "Bob"
#   ..$ age : num 20
#  $ grades:List of 3
#   ..$ math: num 90
#   ..$ eng : num 85
#   ..$ sci : num 92
```

Ex 44: `split()` and Recombine

Problem

- **(a)** `split(mtcars\$mpg, mtcars\$cyl)` — what type?
- **(b)** Compute `mean` per group using `sapply`
- **(c)** Same using `tapply` in one line
- **(d)** `split(1:10, rep(c("odd","even"), 5))`

Sol 44

```r
sp <- split(mtcars$mpg, mtcars$cyl); class(sp)    # (a) list
```

```
# [1] "list"
```

```r
sapply(sp, mean)                                   # (b)
```

```
#        4        6        8
# 26.66364 19.74286 15.10000
```

```r
tapply(mtcars$mpg, mtcars$cyl, mean)               # (c)
```

```
#        4        6        8
# 26.66364 19.74286 15.10000
```

```r
split(1:10, rep(c("odd","even"), 5))               # (d)
```

```
# $even
# [1]  2  4  6  8 10
#
# $odd
# [1] 1 3 5 7 9
```

Ex 45: List Concatenation

Problem

- **(a)** Concatenate `list(a=1)` and `list(b=2)` with `c()`
- **(b)** What happens with `c(list(a=1), 5)`?
- **(c)** Build a list incrementally in a loop: for i=1:5, append `i\textasciicircum{2}`

Sol 45

```r
c(list(a=1), list(b=2))                                  # (a)
```

```
# $a
# [1] 1
#
# $b
# [1] 2
```

```r
c(list(a=1), 5)                                           # (b) 5 becomes element
```

```
# $a
# [1] 1
#
# [[2]]
# [1] 5
```

```r
l <- list()
for (i in 1:5) l[[i]] <- i^2; l                          # (c)
```

```
# [[1]]
# [1] 1
#
# [[2]]
# [1] 4
#
# [[3]]
# [1] 9
```

Ex 46: Conversion Table

Problem

- **(a)** Vector to list: `as.list(1:3)` vs `list(1:3)`. Explain difference
- **(b)** List to data frame: `as.data.frame(list(a=1:3, b=4:6))`
- **(c)** Data frame to matrix: what happens with `as.matrix(data.frame(x=1:3, y=c("a","b","c")))`?
- **(d)** List to vector: why use `unlist` not `as.vector`?

Sol 46

```r
as.list(1:3)    # 3 elements                        # (a)
```

```
# [[1]]
# [1] 1
#
# [[2]]
# [1] 2
#
# [[3]]
# [1] 3
```

```r
list(1:3)       # 1 element (the vector)
```

```
# [[1]]
# [1] 1 2 3
```

```r
as.data.frame(list(a=1:3, b=4:6))                   # (b)
```

```
#   a b
# 1 1 4
# 2 2 5
# 3 3 6
```

```r
as.matrix(data.frame(x=1:3, y=c("a","b","c")))    # (c) all char!
```

```
#      x   y
# [1,] "1" "a"
```

Ex 47: Named List as Lookup Table

Problem

- **(a)** Create a lookup: `grade\_points <- list(A=4.0, B=3.0, C=2.0, D=1.0, F=0)`
- **(b)** Look up GPA for grades `c("A","B","F","A","C")` using `sapply` + `[[`
- **(c)** Compute mean GPA

Sol 47

```r
grade_points <- list(A=4.0, B=3.0, C=2.0, D=1.0, F=0)
g <- c("A","B","F","A","C")
gpa <- sapply(g, function(x) grade_points[[x]])    # (b)
gpa
```

```
# A B F A C
# 4 3 0 4 2
```

```r
mean(gpa)                                          # (c) 2.6
```

```
# [1] 2.6
```

Ex 48: `lapply/sapply` on Lists

Problem

`l <- list(a=1:10, b=rnorm(20), c=c(TRUE,FALSE,TRUE,TRUE)).`

- **(a)** Compute `mean` of each element with `lapply` and `sapply`
- **(b)** Compute `length` of each
- **(c)** Apply custom function: range (max - min)

Sol 48

```r
set.seed(1)
l <- list(a=1:10, b=rnorm(20), c=c(TRUE,FALSE,TRUE,TRUE))
lapply(l, mean); sapply(l, mean)                    # (a)
```

```
# $a
# [1] 5.5
#
# $b
# [1] 0.1905239
#
# $c
# [1] 0.75

#         a         b         c
# 5.5000000 0.1905239 0.7500000
```

```r
sapply(l, length)                                   # (b)
```

```
#  a  b  c
# 10 20  4
```

```r
sapply(l, function(x) diff(range(x)))               # (c)
```

```
#        a        b        c
# 9.000000 3.809981 1.000000
```

Ex 49: `Filter`, `Find`, `Position`

Problem

`l <- list(1:3, "hello", 4:6, TRUE, NULL, c(1.5, 2.5))`.

- **(a)** Keep only numeric elements
- **(b)** Find first character element
- **(c)** Position of first character element
- **(d)** Filter integers from `1:30` that are divisible by both 3 and 7

Sol 49

```r
l <- list(1:3, "hello", 4:6, TRUE, NULL, c(1.5, 2.5))
Filter(is.numeric, l)                                # (a)
```

```
# [[1]]
# [1] 1 2 3
#
# [[2]]
# [1] 4 5 6
#
# [[3]]
# [1] 1.5 2.5
```

```r
Find(is.character, l)                                # (b) "hello"
```

```
# [1] "hello"
```

```r
Position(is.character, l)                            # (c) 2
```

```
# [1] 2
```

```r
Filter(\(x) x%%3==0 & x%%7==0, 1:30)                # (d) 21
```

```
# [1] 21
```

Ex 50: `Reduce()`

Problem

- **(a)** Compute `1+2+3+4+5` using `Reduce("+", 1:5)`. Show accumulated steps
- **(b)** Find intersection of 3 vectors via `Reduce(intersect, ...)`
- **(c)** Merge 3 data frames pairwise using `Reduce`

Sol 50

```r
Reduce("+", 1:5)                                    # (a) 15
```

```
# [1] 15
```

```r
Reduce("+", 1:5, accumulate=TRUE)                   # steps
```

```
# [1]  1  3  6 10 15
```

```r
vecs <- list(c(1,2,3,4), c(2,3,4,5), c(3,4,5,6))
Reduce(intersect, vecs)                             # (b) 3 4
```

```
# [1] 3 4
```

```r
dfs <- list(data.frame(id=1:3, a=10:12),
            data.frame(id=2:4, b=20:22))
Reduce(\(x,y) merge(x,y,by="id",all=TRUE), dfs)   # (c)
```

```
#   id  a  b
# 1  1 10 NA
# 2  2 11 20
# 3  3 12 21
# 4  4 NA 22
```

Ex 51: List Modification Patterns

Problem

- **(a)** Given `l <- list(x=1, y=2, z=3)`, remove `x` and `z` in one operation
- **(b)** Replace all numeric elements with their squares using `lapply`
- **(c)** Deep copy a nested list and modify without affecting original

Sol 51

```r
l <- list(x=1, y=2, z=3)
l[c("x","z")] <- NULL; l                          # (a) only y
```

```
# $y
# [1] 2
```

```r
l2 <- list(a=3, b="hi", c=4)
lapply(l2, function(x) if(is.numeric(x)) x^2 else x) # (b)
```

```
# $a
# [1] 9
#
# $b
# [1] "hi"
#
# $c
# [1] 16
```

```r
orig <- list(inner=list(val=10))                  # (c)
copy <- orig; copy$inner$val <- 99
orig$inner$val  # still 10 (R copies on modify)
```

```
# [1] 10
```

Ex 52: `rapply()` — Recursive Apply

Problem

`l <- list(a=1:3, b=list(c=4:6, d=list(e=7:9)))`.

- **(a)** Apply `sum` to all leaf elements using `rapply(l, sum)`
- **(b)** Use `rapply(l, sum, how="unlist")` vs `how="list"`
- **(c)** Double all values with `rapply(l, function(x) x*2, how="replace")`

Sol 52

```r
l <- list(a=1:3, b=list(c=4:6, d=list(e=7:9)))
rapply(l, sum)                                    # (a)
```

```
#     a   b.c b.d.e
#     6    15    24
```

```r
rapply(l, sum, how="unlist")                      # (b) named vector
```

```
#     a   b.c b.d.e
#     6    15    24
```

```r
rapply(l, sum, how="list")                        # list structure
```

```
# $a
# [1] 6
#
# $b
# $b$c
# [1] 15
#
# $b$d
# $b$d$e
# [1] 24
```

```r
str(rapply(l, \(x) x*2, how="replace"))           # (c) doubled
```

```
# List of 2
```

Matrices (Ex 53–58)

Ex 53: Creating Matrices

Problem

- **(a)** Create 3×4 matrix filled column-wise from `1:12`
- **(b)** Create same matrix filled row-wise
- **(c)** Name rows `r1,r2,r3` and columns `c1,...,c4`. Access `m["r2","c3"]`
- **(d)** Create a 3×3 identity matrix with `diag`

Sol 53

```r
matrix(1:12, nrow=3)                                    # (a)
```

```
#      [,1] [,2] [,3] [,4]
# [1,]    1    4    7   10
# [2,]    2    5    8   11
# [3,]    3    6    9   12
```

```r
matrix(1:12, nrow=3, byrow=TRUE)                        # (b)
```

```
#      [,1] [,2] [,3] [,4]
# [1,]    1    2    3    4
# [2,]    5    6    7    8
# [3,]    9   10   11   12
```

```r
m <- matrix(1:12, 3, 4)
rownames(m) <- paste0("r",1:3); colnames(m) <- paste0("c",1:4)
m["r2","c3"]                                            # (c) 8
```

```
# [1] 8
```

```r
diag(3)                                                 # (d)
```

```
#      [,1] [,2] [,3]
# [1,]    1    0    0
# [2,]    0    1    0
# [3,]    0    0    1
```

Ex 54: Matrix Arithmetic

Problem

`A <- matrix(1:4, 2, 2); B <- matrix(5:8, 2, 2).`

- **(a)** Element-wise `*` vs matrix `%*%`
- **(b)** Compute `t(A)`, `det(A)`, `solve(A)`
- **(c)** Verify `A %*% solve(A)` is the identity
- **(d)** `crossprod(A,B)` vs `t(A) %*% B`

Sol 54

```r
A <- matrix(1:4, 2, 2); B <- matrix(5:8, 2, 2)
A * B; A %*% B                                          # (a)
```

```
#      [,1] [,2]
# [1,]    5   21
# [2,]   12   32

#      [,1] [,2]
# [1,]   23   31
# [2,]   34   46
```

```r
t(A); det(A); solve(A)                                  # (b)
```

```
#      [,1] [,2]
# [1,]    1    2
# [2,]    3    4

# [1] -2

#      [,1] [,2]
# [1,]   -2  1.5
# [2,]    1 -0.5
```

```r
round(A %*% solve(A))                                    # (c) identity
```

```
#      [,1] [,2]
# [1,]    1    0
```

Ex 55: `apply()` on Matrices

Problem

`m <- matrix(1:12, nrow=3)`.

- **(a)** Row sums and column sums via `apply`
- **(b)** Per-column range
- **(c)** Standardize each column (subtract mean, divide by sd)
- **(d)** Compare `apply(m,2,sum)` speed with `colSums(m)`

Sol 55

```r
m <- matrix(1:12, nrow=3)
apply(m, 1, sum); apply(m, 2, sum)                    # (a)
```

```
# [1] 22 26 30

# [1]  6 15 24 33
```

```r
apply(m, 2, range)                                     # (b)
```

```
#      [,1] [,2] [,3] [,4]
# [1,]    1    4    7   10
# [2,]    3    6    9   12
```

```r
apply(m, 2, \(x) (x-mean(x))/sd(x))                   # (c)
```

```
#      [,1] [,2] [,3] [,4]
# [1,]   -1   -1   -1   -1
# [2,]    0    0    0    0
# [3,]    1    1    1    1
```

```r
all.equal(apply(m,2,sum), colSums(m))                  # (d) TRUE
```

```
# [1] TRUE
```

Ex 56: `outer()` Product

Problem

- **(a)** Multiplication table 1..5 $\times$ 1..5 using `outer`
- **(b)** Compute $f(x, y) = x^2 + y^2$ for `x=1:4`, `y=1:3`
- **(c)** All pairwise `paste` of `c("a","b")` and `c("1","2","3")`

Sol 56

```r
outer(1:5, 1:5, "*")                                  # (a)
```

```
#      [,1] [,2] [,3] [,4] [,5]
# [1,]    1    2    3    4    5
# [2,]    2    4    6    8   10
# [3,]    3    6    9   12   15
# [4,]    4    8   12   16   20
# [5,]    5   10   15   20   25
```

```r
outer(1:4, 1:3, \(x,y) x^2 + y^2)                    # (b)
```

```
#      [,1] [,2] [,3]
# [1,]    2    5   10
# [2,]    5    8   13
# [3,]   10   13   18
# [4,]   17   20   25
```

```r
outer(c("a","b"), c("1","2","3"), paste0)             # (c)
```

```
#      [,1] [,2] [,3]
# [1,] "a1" "a2" "a3"
# [2,] "b1" "b2" "b3"
```

Ex 57: Matrix Row/Column Selection

Problem

`m <- matrix(1:12, 3, 4)`.

- **(a)** Extract row 2 as a vector vs as a 1-row matrix (`drop`)
- **(b)** Extract column 3 both ways
- **(c)** Select submatrix: rows 1-2, columns 2-3
- **(d)** Replace all elements > 8 with `NA`

Sol 57

```r
m <- matrix(1:12, 3, 4)
m[2,]; m[2,,drop=FALSE]                                  # (a)
```

```
# [1]  2  5  8 11

#      [,1] [,2] [,3] [,4]
# [1,]    2    5    8   11
```

```r
m[,3]; m[,3,drop=FALSE]                                  # (b)
```

```
# [1] 7 8 9

#      [,1]
# [1,]    7
# [2,]    8
# [3,]    9
```

```r
m[1:2, 2:3]                                              # (c)
```

```
#      [,1] [,2]
# [1,]    4    7
# [2,]    5    8
```

```r
m[m > 8] <- NA; m                                        # (d)
```

```
#      [,1] [,2] [,3] [,4]
# [1,]    1    4    7   NA
```

Ex 58: Linear System

Problem

Solve $2x + 3y = 7,\ 4x - y = 1$.

- **(a)** Set up as `A %*% x = b` and solve with `solve(A, b)`
- **(b)** Verify by multiplying back
- **(c)** Compute eigenvalues of `A`

Sol 58

```r
A <- matrix(c(2,4,3,-1), 2, 2); b <- c(7,1)
x <- solve(A, b); x                                  # (a)
```

```
# [1] 0.7142857 1.8571429
```

```r
A %*% x  # should be c(7,1)                          # (b)
```

```
#      [,1]
# [1,]    7
# [2,]    1
```

```r
eigen(A)$values                                       # (c)
```

```
# [1]  4.274917 -3.274917
```

Programming (Ex 59–84)

Ex 59: Default Arguments

Problem

- **(a)** Write `power(x, n=2)` that computes `x\textasciicircum{n}`. Test with 1 and 2 arguments
- **(b)** Write `greet(name, greeting="Hello")` that returns `"Hello, Alice!"`
- **(c)** What happens if you call `power(n=3)` without `x`?

Sol 59

```r
power <- function(x, n=2) x^n
power(3); power(3, 3); power(x=5, n=2)            # (a)
```

```
# [1] 9

# [1] 27

# [1] 25
```

```r
greet <- function(name, greeting="Hello")
  paste0(greeting, ", ", name, "!")
greet("Alice"); greet("Bob", "Hi")                 # (b)
```

```
# [1] "Hello, Alice!"

# [1] "Hi, Bob!"
```

```r
power(n=3)  # error: x is missing                  # (c)
```

```
# Error in power(n = 3): argument "x" is missing, with no default
```

Ex 60: Functions as Arguments

Problem

- **(a)** Write `apply\_fn(x, fn)` that applies `fn` to `x`. Test with `sqrt`, `log`, `abs`
- **(b)** Extend to `apply\_fn(x, fn, ...)` supporting extra arguments. Test: `apply\_fn(c(1,NA,3), mean, na.rm=TRUE)`

Sol 60

```r
apply_fn <- function(x, fn, ...) fn(x, ...)
apply_fn(c(4,9,16), sqrt)                          # (a)
```

```
# [1] 2 3 4
```

```r
apply_fn(c(1,NA,3), mean, na.rm=TRUE)              # (b) 2
```

```
# [1] 2
```

Ex 61: Implicit Return

Problem

- **(a)** Write two versions of average: one with explicit `return()`, one without
- **(b)** Write `bad\_avg` that assigns to a local variable as last line. Show it returns silently
- **(c)** Capture the invisible result with `x <- bad\_avg(1:10)`

Sol 61

```r
avg_explicit <- function(x) { return(sum(x)/length(x)) }
avg_implicit <- function(x) { sum(x)/length(x) }
avg_explicit(1:10); avg_implicit(1:10)             # (a) same
```

```
# [1] 5.5
```

```
# [1] 5.5
```

```r
bad_avg <- function(x) { result <- sum(x)/length(x) }
bad_avg(1:10)  # no output!                        # (b)
x <- bad_avg(1:10); x  # but value is captured     # (c)
```

```
# [1] 5.5
```

Ex 62: Anonymous Functions

Problem

- **(a)** Use `sapply(1:5, function(x) x\textasciicircum{2+1})`
- **(b)** Rewrite with `\textbackslash\{\(x)}` shorthand
- **(c)** Use anonymous function in `Filter` to keep even numbers from `1:20`
- **(d)** Nested: `sapply(1:3, \textbackslash\{\(i) sapply(1:3, \{}(j) i*j))}`

Sol 62

```r
sapply(1:5, function(x) x^2+1)                    # (a)
```

```
# [1]  2  5 10 17 26
```

```r
sapply(1:5, \(x) x^2+1)                           # (b)
```

```
# [1]  2  5 10 17 26
```

```r
Filter(\(x) x%%2==0, 1:20)                        # (c)
```

```
#  [1]  2  4  6  8 10 12 14 16 18 20
```

```r
sapply(1:3, \(i) sapply(1:3, \(j) i*j))           # (d) 3x3 matrix
```

```
#      [,1] [,2] [,3]
# [1,]    1    2    3
# [2,]    2    4    6
# [3,]    3    6    9
```

Ex 63: Closures

Problem

- **(a)** Write `make\_adder(n)` that returns a function adding `n`. Test `add5 <- make\_adder(5); add5(3)`
- **(b)** Write `make\_counter()` returning a list with `increment`, `get`, `reset` functions using `<<-`
- **(c)** Verify the counter works across multiple calls

Sol 63

```r
make_adder <- function(n) function(x) x + n        # (a)
add5 <- make_adder(5); add5(3)                      # 8
```

```
# [1] 8
```

```r
make_counter <- function() {                        # (b)
  n <- 0
  list(inc = function() {n <<- n+1; n},
       get = function() n,
       reset = function() {n <<- 0})
}
ctr <- make_counter()
ctr$inc(); ctr$inc(); ctr$get()                     # (c) 2
```

```
# [1] 1

# [1] 2

# [1] 2
```

```r
ctr$reset(); ctr$get()                              # 0
```

```
# [1] 0
```

Ex 64: Dot-Prefix Naming

Problem

- **(a)** Define `.helper <- function(x) x+1` and `public <- function(x) .helper(x)*2`
- **(b)** Show `.helper` doesn't appear in `ls()` but does in `ls(all.names=TRUE)`
- **(c)** Name 3 built-in R objects starting with `.`

Sol 64

```r
e <- new.env()
e$.helper <- function(x) x+1
e$public  <- function(x) e$.helper(x)*2
e$public(5)  # 12                                  # (a)
```

```
# [1] 12
```

```r
ls(e); ls(e, all.names=TRUE)                        # (b)
```

```
# [1] "public"

# [1] ".helper" "public"
```

```r
# (c) .Machine, .Platform, .GlobalEnv
```

Ex 65: `if (FALSE)` Idiom

Problem

- **(a)** Wrap code in `if (FALSE) \{ ... \}` and show it doesn't execute
- **(b)** Change to `if (TRUE)` and show it runs
- **(c)** Why is this better than commenting with `\#` for multi-line blocks?

Sol 65

```r
if (FALSE) {                                        # (a)
  cat("This won't print\n")
  x <- 999
}
exists("x")  # from previous chunks, may be TRUE
```

```
# [1] TRUE
```

```r
if (TRUE) { cat("This prints\n") }                  # (b)
```

```
# This prints
```

```r
# (c) Unlike #, the code inside is still parsed by R,
# so syntax errors are caught immediately.
```

Ex 66: Local Helper Functions

Problem

- **(a)** Write `outer\_fn(x)` that defines inner helper `validate(v)` checking `is.numeric(v) \&\& length(v) > 0`, then uses it
- **(b)** Show that `validate` is not accessible outside `outer\_fn`
- **(c)** Why is this pattern useful?

Sol 66

```r
outer_fn <- function(x) {
  validate <- function(v) is.numeric(v) && length(v) > 0
  if (!validate(x)) stop("invalid input")
  mean(x)
}
outer_fn(1:10)                                          # (a) 5.5
```

```
# [1] 5.5
```

```r
outer_fn("abc")                                         # error
```

```
# Error in outer_fn("abc"): invalid input
```

```r
# (b) validate is local, not in global env
# (c) keeps global namespace clean; helper has access to
#     outer function's variables via lexical scoping
```

Ex 67: `if/else` and `ifelse`

Problem

- **(a)** Write scalar `classify(x)` returning "pos", "neg", or "zero" using `if/else`
- **(b)** Vectorize: classify `c(-3,0,5,2,-1)` using `ifelse`
- **(c)** Nested `ifelse` to map scores to grades: ≥ 90 A, ≥ 80 B, ≥ 70 C, else F

Sol 67

```r
classify <- function(x) {                          # (a)
  if (x > 0) "pos" else if (x < 0) "neg" else "zero"
}
classify(5); classify(-1); classify(0)
```

```
# [1] "pos"

# [1] "neg"

# [1] "zero"
```

```r
ifelse(c(-3,0,5,2,-1)>0, "pos",
  ifelse(c(-3,0,5,2,-1)<0, "neg", "zero"))         # (b)
```

```
# [1] "neg"  "zero" "pos"  "pos"  "neg"
```

```r
s <- c(95,82,71,65,90)                             # (c)
ifelse(s>=90,"A",ifelse(s>=80,"B",ifelse(s>=70,"C","F")))
```

```
# [1] "A" "B" "C" "F" "A"
```

Ex 68: `switch()`

Problem

- **(a)** Write `calc(op, a, b)` using `switch(op, "+"=a+b, "-"=a-b, "*"=a*b, "/"=a/b, stop("unknown"))`
- **(b)** Test with all 4 operators and an invalid one
- **(c)** Write `day\_type(day)` classifying weekdays vs weekends

Sol 68

```r
calc <- function(op, a, b)                          # (a)
  switch(op, "+"=a+b, "-"=a-b, "*"=a*b, "/"=a/b,
         stop("unknown op"))
calc("+",3,4); calc("*",5,6); calc("/",10,3)      # (b)
```

```
# [1] 7

# [1] 30

# [1] 3.333333
```

```r
calc("^",2,3)  # error
```

```
# Error in calc("^", 2, 3): unknown op
```

```r
day_type <- function(d) switch(d, Monday=,Tuesday=,# (c)
  Wednesday=,Thursday=,Friday="Weekday",
  Saturday=,Sunday="Weekend","Unknown")
day_type("Monday"); day_type("Sunday")
```

```
# [1] "Weekday"

# [1] "Weekend"
```

Ex 69: && / || vs & / |

Problem

- **(a)** Show `\&\&` short-circuits: `FALSE \&\& stop("boom")` doesn't error
- **(b)** Show `\&` doesn't short-circuit: `c(FALSE,TRUE) \& c(TRUE,TRUE)` is vectorized
- **(c)** Write safe null-check: `if (!is.null(x) \&\& x > 0)` and explain why `\&\&` is required

Sol 69

```r
FALSE && stop("boom")  # no error: short-circuit   # (a)
```

```
# [1] FALSE
```

```r
c(FALSE,TRUE) & c(TRUE,TRUE)                       # (b) vectorized
```

```
# [1] FALSE  TRUE
```

```r
x <- NULL
if (!is.null(x) && x > 0) cat("pos") else cat("safe") # (c)
```

```
# safe
```

```r
# && stops after !is.null(x) is FALSE; & would try x>0
```

Ex 70: `for` Loop — FizzBuzz

Problem

- **(a)** Classic FizzBuzz 1-20: print "Fizz" if divisible by 3, "Buzz" if by 5, "FizzBuzz" if both, else the number
- **(b)** Vectorized version using `ifelse` (no loop)
- **(c)** Compare with `paste0` trick

Sol 70

```r
for (i in 1:20) {                                    # (a)
  if (i%%15==0) cat("FizzBuzz ")
  else if (i%%3==0) cat("Fizz ")
  else if (i%%5==0) cat("Buzz ")
  else cat(i, "")
}
```

```
# 1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz 16 17 Fizz 19 Buzz
```

```r
cat("\n")
```

```r
x <- 1:20                                            # (b)
ifelse(x%%15==0,"FizzBuzz",ifelse(x%%3==0,"Fizz",
  ifelse(x%%5==0,"Buzz",as.character(x))))
```

```
#  [1] "1"        "2"        "Fizz"     "4"        "Buzz"     "Fizz"
#  [7] "7"        "8"        "Fizz"     "Buzz"     "11"       "Fizz"
# [13] "13"       "14"       "FizzBuzz" "16"       "17"       "Fizz"
# [19] "19"       "Buzz"
```

Ex 71: `while` Loop — Newton's Method

Problem

Implement Newton's method for $\sqrt{a}$: $x_{n+1} = \frac{1}{2}(x_n + a/x_n)$.

- **(a)** Write `my\_sqrt(a, tol=1e-10)` using `while`
- **(b)** Test on 2, 9, 100
- **(c)** Count iterations needed for a=2

Sol 71

```r
my_sqrt <- function(a, tol=1e-10) {
  x <- a; n <- 0
  while (TRUE) {
    x_new <- 0.5 * (x + a/x); n <- n + 1
    if (abs(x_new - x) < tol) break
    x <- x_new
  }
  list(root=x_new, iterations=n)
}
my_sqrt(2); my_sqrt(9)$root; my_sqrt(100)$root     # (a)(b)
```

```
# $root
# [1] 1.414214
#
# $iterations
# [1] 5

# [1] 3

# [1] 10
```

```r
my_sqrt(2)$iterations                              # (c)
```

```
# [1] 5
```

Ex 72: `repeat/break` — Collatz

Problem

Collatz: if n even, $n/2$; if odd, $3n + 1$. Conjecture: always reaches 1.

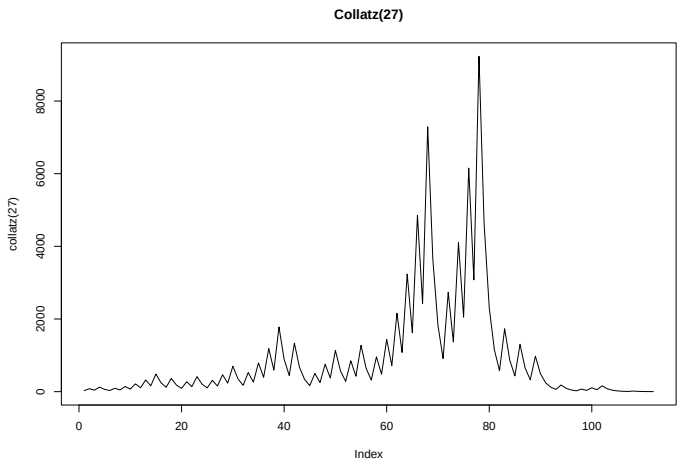
- **(a)** Write `collatz(n)` returning the sequence using `repeat/break`
- **(b)** Find the starting number 1–100 with the longest sequence
- **(c)** Plot the sequence for `n=27`

Sol 72

```
collatz <- function(n) {
  seq <- n
  repeat { if (n==1) break; n <- if(n%%2==0) n/2 else 3*n+1
           seq <- c(seq, n) }
  seq
}
lengths <- sapply(1:100, \(n) length(collatz(n)))  # (b)
which.max(lengths)  # starting number
```

```
# [1] 97
```

```
plot(collatz(27), type="l", main="Collatz(27)")    # (c)
```



Ex 73: `next` — Filtered Sum

Problem

- **(a)** Sum integers 1-100 that are NOT multiples of 7, using `for+next`
- **(b)** Same with vectorized filtering (no loop)
- **(c)** Sum only prime numbers 1-50. Write `is\_prime(n)` helper

Sol 73

```r
s <- 0; for(i in 1:100) {if(i%%7==0) next; s<-s+i}; s # (a)
```

```
# [1] 4315
```

```r
sum((1:100)[(1:100)%%7 != 0])                          # (b) same
```

```
# [1] 4315
```

```r
is_prime <- function(n) {
  if (n < 2) return(FALSE)
  if (n == 2) return(TRUE)
  all(n %% 2:floor(sqrt(n)) != 0)
}
sum(Filter(is_prime, 1:50))                            # (c)
```

```
# [1] 325
```

Ex 74: Vectorization Speed

Problem

- **(a)** Write loop version of `sum(x\textasciicircum{2)}` for `x <- 1:1e5`
- **(b)** Compare `system.time` with vectorized `sum(x\textasciicircum{2)}`
- **(c)** Pre-allocate vs grow-in-loop: time `v <- c(); for(...) v <- c(v,i)` vs `v <- numeric(n); for(...) v[i] <- i`

Sol 74

```r
n <- 1e5; x <- 1:n
loop_sum <- function(x) {s<-0; for(v in x) s<-s+v^2; s}
system.time(loop_sum(x))                              # (a) slow
```

```
#    user  system elapsed
#   0.004   0.001   0.004
```

```r
system.time(sum(x^2))                                 # (b) fast
```

```
#    user  system elapsed
#       0       0       0
```

```r
system.time({v<-c(); for(i in 1:n) v<-c(v,i)})    # (c) very slow
```

```
#    user  system elapsed
#   4.640   0.036   4.676
```

```r
system.time({v<-numeric(n); for(i in 1:n) v[i]<-i}) # faster
```

```
#    user  system elapsed
#   0.004   0.000   0.004
```

Ex 75: Sieve of Eratosthenes

Problem

- **(a)** Implement `sieve(n)` returning all primes up to `n` using a logical vector and nested loops
- **(b)** Test with `sieve(50)`
- **(c)** How many primes below 1000?

Sol 75

```r
sieve <- function(n) {
  is_p <- rep(TRUE, n); is_p[1] <- FALSE
  for (i in 2:floor(sqrt(n)))
    if (is_p[i]) is_p[seq(i*2, n, i)] <- FALSE
  which(is_p)
}
sieve(50)                                          # (b)
```

```
#  [1]  2  3  5  7 11 13 17 19 23 29 31 37 41 43 47
```

```r
length(sieve(1000))                                # (c) 168
```

```
# [1] 168
```

Ex 76: Recursive Factorial and Fibonacci

Problem

- **(a)** Write `my\_fact(n)` using `Recall`
- **(b)** Write `fib(n)` using `Recall`
- **(c)** Compute `sapply(1:10, fib)`
- **(d)** Why is naive recursive `fib` slow for large `n`?

Sol 76

```r
my_fact <- function(n) {                                # (a)
  if (n <= 1) return(1); n * Recall(n-1)
}
my_fact(5); my_fact(10)
```

```
# [1] 120

# [1] 3628800
```

```r
fib <- function(n) {                                    # (b)
  if (n <= 2) return(1); Recall(n-1) + Recall(n-2)
}
sapply(1:10, fib)                                       # (c)
```

```
#  [1]  1  1  2  3  5  8 13 21 34 55
```

```r
# (d) exponential time: each call branches into 2
```

Ex 77: `match.arg` and `missing`

Problem

- **(a)** Write `summary\_stat(x, type=c("mean","median","sd"))` using `match.arg`. Test partial matching
- **(b)** Write `f(x, y)` that checks `missing(y)` and uses default `y=0`
- **(c)** Show `nargs()` inside a function

Sol 77

```r
summary_stat <- function(x, type=c("mean","median","sd")) {
  type <- match.arg(type)
  switch(type, mean=mean(x), median=median(x), sd=sd(x))
}
summary_stat(1:10, "mea")                          # (a) mean
```

```
# [1] 5.5
```

```r
f <- function(x, y) {
  if (missing(y)) y <- 0; x + y
}
f(5); f(5, 3)                                      # (b)
```

```
# [1] 5

# [1] 8
```

```r
g <- function(a,b,c) nargs(); g(1); g(1,2,3)      # (c)
```

```
# [1] 1

# [1] 3
```

Ex 78: `invisible` and Side Effects

Problem

- **(a)** Write `my\_print(x)` that cats and returns `invisible(x)`. Show it doesn't print the return
- **(b)** Capture the invisible return
- **(c)** Write `log\_and\_return(x)` that prints a message and returns `invisible(NULL)`

Sol 78

```r
my_print <- function(x) { cat("Value:", x, "\n"); invisible(x) }
my_print(42)          # prints message, no [1] 42   # (a)
```

```
# Value: 42
```

```r
val <- my_print(42); val                            # (b) captured
```

```
# Value: 42

# [1] 42
```

```r
log_return <- function(x) {
  cat("Processed:", x, "\n"); invisible(NULL)
}
r <- log_return(5); is.null(r)                      # (c) TRUE
```

```
# Processed: 5

# [1] TRUE
```

Ex 79: `on.exit` with `add=TRUE`

Problem

- **(a)** Write function that sets `options(digits=3)`, uses `on.exit` to restore
- **(b)** Write function with two `on.exit(add=TRUE)` calls. Show both run
- **(c)** Show cleanup runs even on error

Sol 79

```r
show_digits <- function() {
  old <- options(digits=3)
  on.exit(options(old))
  cat("Inside:", getOption("digits"), "\n")
}
show_digits(); cat("Outside:", getOption("digits"), "\n") # (a)
```

```
# Inside: 3

# Outside: 7
```

```r
two_exits <- function() {                          # (b)
  on.exit(cat("exit1\n"), add=TRUE)
  on.exit(cat("exit2\n"), add=TRUE)
  cat("body\n")
}
two_exits()
```

```
# body
# exit1
# exit2
```

```r
tryCatch({                                          # (c)
  f <- function() { on.exit(cat("cleanup!\n")); stop("err") }
  f()
}, error = function(e) cat("caught:", e$message, "\n"))
```

Ex 80: `Vectorize()`

Problem

- **(a)** Write scalar `classify(x)` using `if/else` (not vectorized)
- **(b)** Use `Vectorize` to create vector version. Test on `c(-2, 0, 5)`
- **(c)** Why is `Vectorize` just a wrapper around `mapply`?

Sol 80

```r
classify <- function(x) {
  if (x > 0) "pos" else if (x < 0) "neg" else "zero"
}
classify_v <- Vectorize(classify)                    # (b)
classify_v(c(-2, 0, 5))
```

```
# [1] "neg"  "zero" "pos"
```

```r
# (c) Vectorize calls mapply internally, applying the
# scalar function element-wise. Not truly vectorized (still loops).
```

Ex 81: Native Pipe |>

Problem

- **(a)** Rewrite `round(mean(abs(c(-3,4,-1,2))), 2)` using `|>`
- **(b)** Pipe `mtcars` through `subset` then `head`
- **(c)** Use _ placeholder: `mtcars |> lm(mpg \textasciitilde\{\ wt, data = _)}`

Sol 81

```r
c(-3,4,-1,2) |> abs() |> mean() |> round(2)        # (a)
```

```
# [1] 2.5
```

```r
mtcars |> subset(cyl == 4) |> head(3)              # (b)
```

```
#             mpg cyl  disp hp drat   wt  qsec vs am gear carb
# Datsun 710 22.8   4 108.0 93 3.85 2.32 18.61  1  1    4    1
# Merc 240D  24.4   4 146.7 62 3.69 3.19 20.00  1  0    4    2
# Merc 230   22.8   4 140.8 95 3.92 3.15 22.90  1  0    4    2
```

```r
mtcars |> lm(mpg ~ wt, data = _) |>                # (c)
  coef()
```

```
# (Intercept)          wt
#   37.285126   -5.344472
```

Ex 82: `do.call`

Problem

- **(a)** Use `do.call(paste, c(as.list(c("a","b","c")), sep="-"))`
- **(b)** Use `do.call(rbind, list\_of\_dfs)` to stack 3 data frames
- **(c)** Build a call to `seq` dynamically from a list of arguments

Sol 82

```r
do.call(paste, c(as.list(c("a","b","c")), sep="-")) # (a)
```

```
# [1] "a-b-c"
```

```r
dfs <- list(data.frame(x=1,y=2), data.frame(x=3,y=4))
do.call(rbind, dfs)                                  # (b)
```

```
#   x y
# 1 1 2
# 2 3 4
```

```r
do.call(seq, list(from=1, to=10, by=2))            # (c)
```

```
# [1] 1 3 5 7 9
```

Ex 83: `tryCatch` and `try`

Problem

- **(a)** Use `tryCatch(log("a"), error=function(e) NA)`
- **(b)** Loop over `list(1,"a",4)`, wrapping each in `try`. Keep only successes
- **(c)** Write `safe\_div(a,b)` that stops on `b==0` and test with `tryCatch`

Sol 83

```r
tryCatch(log("a"), error=function(e) NA)            # (a)
```

```
# [1] NA
```

```r
inputs <- list(1, "a", 4)                           # (b)
results <- lapply(inputs, \(x) try(sqrt(x), silent=TRUE))
Filter(\(r) !inherits(r,"try-error"), results)
```

```
# [[1]]
# [1] 1
#
# [[2]]
# [1] 2
```

```r
safe_div <- function(a,b) {if(b==0) stop("div0!"); a/b}
tryCatch(safe_div(10,0), error=function(e) Inf)     # (c)
```

```
# [1] Inf
```

Ex 84: stopifnot and warning

Problem

- **(a)** Write `my\_mean(x)` with `stopifnot(is.numeric(x), length(x)>0)`
- **(b)** Write `safe\_sqrt(x)` that issues `warning` for negative input and returns `sqrt(abs(x))`
- **(c)** Use `suppressWarnings` to silence it

Sol 84

```r
my_mean <- function(x) {                          # (a)
  stopifnot(is.numeric(x), length(x) > 0)
  sum(x)/length(x)
}
my_mean(1:5); my_mean("abc")
```

```
# [1] 3

# Error in my_mean("abc"): is.numeric(x) is not TRUE
```

```r
safe_sqrt <- function(x) {                        # (b)
  if (any(x < 0)) warning("Negative values!")
  sqrt(abs(x))
}
safe_sqrt(c(4, -9, 16))
```

```
# [1] 2 3 4
```

```r
suppressWarnings(safe_sqrt(-1))                   # (c)
```

```
# [1] 1
```

Apply Family (Ex 85–96)

Ex 85: `apply` — Row and Column

Problem

`m <- matrix(c(3,1,4,1,5,9,2,6,5,3,5,8), nrow=3)`.

- **(a)** Column means via `apply`
- **(b)** Row ranges
- **(c)** Apply custom function returning a named vector per column
- **(d)** Apply on a data frame's numeric columns

Sol 85

```r
m <- matrix(c(3,1,4,1,5,9,2,6,5,3,5,8), nrow=3)
apply(m, 2, mean)                                    # (a)
```

```
# [1] 2.666667 5.000000 4.333333 5.333333
```

```r
apply(m, 1, range)                                   # (b) 2x3 matrix
```

```
#      [,1] [,2] [,3]
# [1,]    1    1    4
# [2,]    3    6    9
```

```r
apply(m, 2, \(x) c(mu=mean(x), sd=sd(x)))            # (c)
```

```
#        [,1] [,2]     [,3]     [,4]
# mu 2.666667    5 4.333333 5.333333
# sd 1.527525    4 2.081666 2.516611
```

```r
apply(mtcars[1:3, 1:4], 2, mean)                     # (d)
```

```
#        mpg        cyl       disp         hp
#  21.600000   5.333333 142.666667 104.333333
```

Ex 86: `lapply` and `sapply`

Problem

```
l <- list(a=1:10, b=seq(0,1,by=0.1), c=c(T,F,T,T,F)).
```

- **(a)** `sapply(l, length)`
- **(b)** `sapply(l, range)` — what shape?
- **(c)** `vapply(l, mean, numeric(1))`
- **(d)** Why prefer `vapply` over `sapply`?

Sol 86

```r
l <- list(a=1:10, b=seq(0,1,by=0.1), c=c(T,F,T,T,F))
sapply(l, length)                                    # (a) named vector
```

```
#  a  b  c
# 10 11  5
```

```r
sapply(l, range)                                     # (b) 2x3 matrix
```

```
#       a b c
# [1,]  1 0 0
# [2,] 10 1 1
```

```r
vapply(l, mean, numeric(1))                          # (c) named vector
```

```
#   a   b   c
# 5.5 0.5 0.6
```

```r
# (d) vapply guarantees return type; sapply may return list
# unexpectedly if elements have different lengths
```

Ex 87: `tapply`

Problem

- **(a)** Mean `mpg` by `cyl` in `mtcars`
- **(b)** Two-way: mean `mpg` by `cyl` $\times$ `am`
- **(c)** Use `tapply` on `ToothGrowth`: mean `len` by `supp` and `dose`

Sol 87

```r
tapply(mtcars$mpg, mtcars$cyl, mean)                # (a)
```

```
#        4        6        8
# 26.66364 19.74286 15.10000
```

```r
tapply(mtcars$mpg, list(mtcars$cyl, mtcars$am), mean) # (b)
```

```
#        0        1
# 4 22.900 28.07500
# 6 19.125 20.56667
# 8 15.050 15.40000
```

```r
tapply(ToothGrowth$len,
  list(ToothGrowth$supp, ToothGrowth$dose), mean)  # (c)
```

```
#      0.5     1     2
# OJ 13.23 22.70 26.06
# VC  7.98 16.77 26.14
```

Ex 88: `mapply` and `Map`

Problem

- **(a)** `mapply(function(x,y) x+y, 1:5, 10:14)`
- **(b)** `mapply(rep, 1:4, 4:1)`
- **(c)** `Map("+", 1:3, 10:12)` — what type?
- **(d)** Use `Map` to apply `paste` element-wise to two vectors

Sol 88

```r
mapply(function(x,y) x+y, 1:5, 10:14)              # (a)
```

```
# [1] 11 13 15 17 19
```

```r
mapply(rep, 1:4, 4:1)                               # (b) list
```

```
# [[1]]
# [1] 1 1 1 1
#
# [[2]]
# [1] 2 2 2
#
# [[3]]
# [1] 3 3
#
# [[4]]
# [1] 4
```

```r
Map("+", 1:3, 10:12)                                # (c) list
```

```
# [[1]]
# [1] 11
#
# [[2]]
# [1] 13
#
# [[3]]
```

Ex 89: `Reduce` Applications

Problem

- **(a)** Factorial via `Reduce("*", 1:6)`
- **(b)** Cumulative products via `accumulate=TRUE`
- **(c)** Find the common elements of 4 vectors using `Reduce(intersect, ...)`

Sol 89

```r
Reduce("*", 1:6)                                    # (a) 720
```

```
# [1] 720
```

```r
Reduce("*", 1:6, accumulate=TRUE)                   # (b)
```

```
# [1]   1   2   6  24 120 720
```

```r
Reduce(intersect, list(1:10, 3:12, 5:15, 7:20))   # (c) 7:10
```

```
# [1]  7  8  9 10
```

Ex 90: Functional Pipeline

Problem

Process `x <- c(-5, 3, -1, 8, -2, 7)`:

- **(a)** Filter positive → square → sum. Do with nested calls
- **(b)** Same with pipe `|>`
- **(c)** Same with `Reduce` composing 3 functions

Sol 90

```r
x <- c(-5, 3, -1, 8, -2, 7)
sum(x[x>0]^2)                                   # (a) nested
```

```
# [1] 122
```

```r
x[x>0] |> (\(v) v^2)() |> sum()                  # (b) pipe
```

```
# [1] 122
```

```r
fns <- list(\(v) v[v>0], \(v) v^2, sum)
Reduce(\(v,f) f(v), fns, init=x)                 # (c)
```

```
# [1] 122
```

Ex 91: `tapply` + `aggregate` Pipeline

Problem

Using `iris`:

- **(a)** Mean `Sepal.Length` by `Species` with `tapply`
- **(b)** Same with `aggregate`
- **(c)** Mean of all numeric columns by `Species` with `aggregate(. \textasciitilde\{\` Species, ...)}

Sol 91

```r
tapply(iris$Sepal.Length, iris$Species, mean)       # (a)
```

```
#     setosa versicolor  virginica
#      5.006      5.936      6.588
```

```r
aggregate(Sepal.Length ~ Species, data=iris, mean)  # (b)
```

```
#      Species Sepal.Length
# 1     setosa        5.006
# 2 versicolor        5.936
# 3  virginica        6.588
```

```r
aggregate(. ~ Species, data=iris, mean)             # (c)
```

```
#      Species Sepal.Length Sepal.Width Petal.Length Petal.Width
# 1     setosa        5.006       3.428        1.462       0.246
# 2 versicolor        5.936       2.770        4.260       1.326
# 3  virginica        6.588       2.974        5.552       2.026
```

Ex 92: Memoization via Closure

Problem

- **(a)** Write `make\_memo(fn)` that returns a memoized version storing results in a list environment
- **(b)** Memoize `fib`
- **(c)** Show second call is instant

Sol 92

```r
make_memo <- function(fn) {
  cache <- list()
  function(...) {
    key <- paste(..., sep=",")
    if (is.null(cache[[key]])) cache[[key]] <<- fn(...)
    cache[[key]]
  }
}
slow_fn <- function(x) { Sys.sleep(0.1); x^2 }
fast_fn <- make_memo(slow_fn)                          # (b)
system.time(fast_fn(5))                                # slow
```

```
#    user  system elapsed
#     0.0     0.0     0.1
```

```r
system.time(fast_fn(5))                                # (c) instant
```

```
#    user  system elapsed
#       0       0       0
```

Ex 93: `lapply` with Index

Problem

- **(a)** Given `l <- list(a=1:3, b=4:6)`, use `lapply(seq\_along(l), ...)` to create `"a: 1 2 3"`, `"b: 4 5 6"`
- **(b)** Same with `Map` and `names(l)`
- **(c)** Same with `mapply`

Sol 93

```r
l <- list(a=1:3, b=4:6)                                # (a)
lapply(seq_along(l), \(i)
  paste0(names(l)[i], ": ", paste(l[[i]], collapse=" ")))
```

```
# [[1]]
# [1] "a: 1 2 3"
#
# [[2]]
# [1] "b: 4 5 6"
```

```r
Map(\(nm, v) paste0(nm, ": ", paste(v,collapse=" ")), # (b)
    names(l), l)
```

```
# $a
# [1] "a: 1 2 3"
#
# $b
# [1] "b: 4 5 6"
```

```r
mapply(\(nm,v) paste0(nm,": ",paste(v,collapse=" ")), # (c)
       names(l), l)
```

```
#          a          b
# "a: 1 2 3" "b: 4 5 6"
```

Ex 94: Nested `lapply`

Problem

- **(a)** Generate a 3×3 multiplication table using `lapply(1:3, \textbackslash\{\(i)` sapply(1:3, \{}(j) i*j))}
- **(b)** Convert to a matrix with `do.call(rbind, ...)`
- **(c)** Compare with `outer(1:3, 1:3)`

Sol 94

```r
l <- lapply(1:3, \(i) sapply(1:3, \(j) i*j))        # (a)
do.call(rbind, l)                                   # (b)
```

```
#      [,1] [,2] [,3]
# [1,]    1    2    3
# [2,]    2    4    6
# [3,]    3    6    9
```

```r
outer(1:3, 1:3)                                     # (c) same
```

```
#      [,1] [,2] [,3]
# [1,]    1    2    3
# [2,]    2    4    6
# [3,]    3    6    9
```

Ex 95: Map for Parallel Processing

Problem

`prices <- list(c(100,102,98), c(50,55,52), c(200,210,205)). names_v <- c("AAPL", "GOOG", "MSFT").`

- **(a)** Compute return `diff(p)/p[-length(p)]` for each using `Map`
- **(b)** Compute mean return per stock

Sol 95

```r
prices <- list(c(100,102,98), c(50,55,52), c(200,210,205))
names_v <- c("AAPL","GOOG","MSFT")
returns <- Map(\(p) diff(p)/p[-length(p)], prices) # (a)
names(returns) <- names_v
sapply(returns, mean)                              # (b)
```

```
#         AAPL         GOOG         MSFT
# -0.009607843  0.022727273  0.013095238
```

Ex 96: `Filter` + `Reduce` Combo

Problem

- **(a)** From `1:100`, filter multiples of 3, then sum with `Reduce`
- **(b)** From a list of vectors, keep only those with length > 3, then find their common intersection
- **(c)** Compare with direct `sum(Filter(...))`

Sol 96

```r
Reduce("+", Filter(\(x) x%%3==0, 1:100))          # (a)
```

```
# [1] 1683
```

```r
vecs <- list(1:10, 3:8, 2:15, 5:12, 4:20)
long <- Filter(\(v) length(v)>3, vecs)
Reduce(intersect, long)                           # (b) 5:8
```

```
# [1] 5 6 7 8
```

```r
sum(Filter(\(x) x%%3==0, 1:100))                   # (c) same as (a)
```

```
# [1] 1683
```

Strings and Regex (Ex 97–110)

Ex 97: Basic String Functions

Problem

`x <- c("  Hello World  ", "R Programming", "data SCIENCE").`

- **(a)** Trim whitespace; count chars of each
- **(b)** Convert to uppercase, lowercase
- **(c)** `startsWith` / `endsWith` tests
- **(d)** Extract first 4 chars using `substr`

Sol 97

```r
x <- c("  Hello World  ","R Programming","data SCIENCE")
xt <- trimws(x); nchar(xt)                          # (a)
```

```
# [1] 11 13 12
```

```r
toupper(xt); tolower(xt)                            # (b)
```

```
# [1] "HELLO WORLD"   "R PROGRAMMING" "DATA SCIENCE"

# [1] "hello world"   "r programming" "data science"
```

```r
startsWith(xt, "Hello"); endsWith(xt, "SCIENCE")   # (c)
```

```
# [1]  TRUE FALSE FALSE

# [1] FALSE FALSE  TRUE
```

```r
substr(xt, 1, 4)                                    # (d)
```

```
# [1] "Hell" "R Pr" "data"
```

Ex 98: paste and sprintf

Problem

- **(a)** `paste("item", 1:5, sep="\_")`
- **(b)** Same with `collapse=", "`
- **(c)** `sprintf("Name: %s, Score: %.1f", "Alice", 95.678)`
- **(d)** Zero-pad: `sprintf("%03d", 1:5)`

Sol 98

```r
paste("item", 1:5, sep="_")                          # (a)
```

```
# [1] "item_1" "item_2" "item_3" "item_4" "item_5"
```

```r
paste("item", 1:5, sep="_", collapse=", ")           # (b)
```

```
# [1] "item_1, item_2, item_3, item_4, item_5"
```

```r
sprintf("Name: %s, Score: %.1f", "Alice", 95.678)  # (c)
```

```
# [1] "Name: Alice, Score: 95.7"
```

```r
sprintf("%03d", 1:5)                                 # (d)
```

```
# [1] "001" "002" "003" "004" "005"
```

Ex 99: `strsplit`

Problem

- **(a)** Split `"a.b.c.d"` by `"."` (note the escape!)
- **(b)** Split `"hello world foo"` by space
- **(c)** Split each character of `"ABCDE"`
- **(d)** Split `c("a-b", "c-d-e")` — what shape is the result?

Sol 99

```r
strsplit("a.b.c.d", "\\.")                          # (a)
```

```
# [[1]]
# [1] "a" "b" "c" "d"
```

```r
strsplit("hello world foo", " ")                    # (b)
```

```
# [[1]]
# [1] "hello" "world" "foo"
```

```r
strsplit("ABCDE", "")                               # (c)
```

```
# [[1]]
# [1] "A" "B" "C" "D" "E"
```

```r
strsplit(c("a-b","c-d-e"), "-")                     # (d) list of 2
```

```
# [[1]]
# [1] "a" "b"
#
# [[2]]
# [1] "c" "d" "e"
```

Ex 100: grep and grepl

Problem

```
w <- c("statistics","computation","algorithm","distribution","estimation").
```

- **(a)** Words containing "tion"
- **(b)** Words starting with vowel
- **(c)** Words of ≥ 12 characters
- **(d)** grepl to create logical mask

Sol 100

```r
w <- c("statistics","computation","algorithm",
       "distribution","estimation")
grep("tion", w, value=TRUE)                          # (a)
```

```
# [1] "computation"  "distribution" "estimation"
```

```r
grep("^[aeiou]", w, value=TRUE)                      # (b)
```

```
# [1] "algorithm"  "estimation"
```

```r
w[nchar(w) >= 12]                                    # (c)
```

```
# [1] "distribution"
```

```r
grepl("tion", w)                                     # (d)
```

```
# [1] FALSE  TRUE FALSE  TRUE  TRUE
```

Ex 101: sub and gsub

Problem

- **(a)** Replace first space with `"\_"` in `"hello world foo"`
- **(b)** Replace all spaces
- **(c)** Remove all digits from `"abc123def456"`
- **(d)** Compress multiple spaces to one

Sol 101

```r
sub(" ", "_", "hello world foo")                    # (a) first only
```

```
# [1] "hello_world foo"
```

```r
gsub(" ", "_", "hello world foo")                   # (b) all
```

```
# [1] "hello_world_foo"
```

```r
gsub("[0-9]", "", "abc123def456")                   # (c)
```

```
# [1] "abcdef"
```

```r
gsub(" +", " ", "too   many    spaces")             # (d)
```

```
# [1] "too many spaces"
```

Ex 102: Capture Groups

Problem

- **(a)** Swap first/last name: `sub("(\textbackslash\{\\{}w+) (\{}\{}w+)“,”\{}\{}2, \{}\{}1”, “John Smith”)}`
- **(b)** Extract domain from email: `sub(".*@", "", "user@example.com")`
- **(c)** Extract year from dates using capture group

Sol 102

```r
sub("(\\w+) (\\w+)", "\\2, \\1",
    c("John Smith","Jane Doe"))                    # (a)
```

```
# [1] "Smith, John" "Doe, Jane"
```

```r
sub(".*@", "", c("user@example.com","a@b.org"))    # (b)
```

```
# [1] "example.com" "b.org"
```

```r
sub("(\\d{4})-\\d{2}-\\d{2}", "\\1",
    c("2024-01-15","2023-12-31"))                  # (c) year
```

```
# [1] "2024" "2023"
```

Ex 103: `regexpr` and `gregexpr`

Problem

`x <- "Phone: 02-1234-5678, Fax: 04-8765-4321".`

- **(a)** Extract all phone numbers with `gregexpr` + `regmatches`
- **(b)** Extract first match only with `regexpr`
- **(c)** Extract all words with `\textbackslash\{\\{}b\{}\{}w+\{}\{}b}`

Sol 103

```r
x <- "Phone: 02-1234-5678, Fax: 04-8765-4321"
m <- gregexpr("[0-9]+-[0-9]+-[0-9]+", x)
regmatches(x, m)                                    # (a)
```

```
# [[1]]
# [1] "02-1234-5678" "04-8765-4321"
```

```r
m1 <- regexpr("[0-9]+-[0-9]+-[0-9]+", x)
regmatches(x, m1)                                   # (b)
```

```
# [1] "02-1234-5678"
```

```r
regmatches(x, gregexpr("\\b\\w+\\b", x))           # (c)
```

```
# [1] "Phone" "02"    "1234"  "5678"  "Fax"   "04"    "8765"  "4321"
```

Ex 104: Greedy vs Lazy

Problem

`x <- "<b>bold</b> and <i>italic</i>".`

- **(a)** `gregexpr("<.*>", x)` + `regmatches` — what matches? (greedy)
- **(b)** `gregexpr("<.*?>", x)` — what matches? (lazy)
- **(c)** Explain the difference

Sol 104

```r
x <- "<b>bold</b> and <i>italic</i>"
regmatches(x, gregexpr("<.*>", x))                # (a) one big match
```

```
# [[1]]
# [1] "<b>bold</b> and <i>italic</i>"
```

```r
regmatches(x, gregexpr("<.*?>", x))               # (b) 4 tags
```

```
# [[1]]
# [1] "<b>"  "</b>" "<i>"  "</i>"
```

```r
# (c) greedy: .* matches as much as possible (all the way
#     to last >); lazy: .*? matches as little as possible
```

Ex 105: Anchors and Quantifiers

Problem

`x <- c("cat","concatenate","scat","catalog")`.

- **(a)** Starts with `cat`
- **(b)** Ends with `cat`
- **(c)** Exactly `cat`
- **(d)** `c("a","ab","abb","abbb")`: match `ab+` vs `ab*` vs `ab?`

Sol 105

```r
x <- c("cat","concatenate","scat","catalog")
grep("^cat", x, value=TRUE)                          # (a)
```

```
# [1] "cat"     "catalog"
```

```r
grep("cat$", x, value=TRUE)                          # (b)
```

```
# [1] "cat"  "scat"
```

```r
grep("^cat$", x, value=TRUE)                         # (c)
```

```
# [1] "cat"
```

```r
y <- c("a","ab","abb","abbb")
grep("ab+", y, value=TRUE); grep("ab*", y, value=TRUE) # (d)
```

```
# [1] "ab"   "abb"  "abbb"
```

```
# [1] "a"    "ab"   "abb"  "abbb"
```

Ex 106: POSIX Classes

Problem

`x <- c("Hello!", "123", "abc", "A1!").`

- **(a)** Contains punctuation
- **(b)** All digits
- **(c)** Contains only alphabetic
- **(d)** All alphanumeric

Sol 106

```r
x <- c("Hello!", "123", "abc", "A1!")
grep("[[:punct:]]", x, value=TRUE)                # (a)
```

```
# [1] "Hello!" "A1!"
```

```r
grep("^[[:digit:]]+$", x, value=TRUE)             # (b)
```

```
# [1] "123"
```

```r
grep("^[[:alpha:]]+$", x, value=TRUE)             # (c)
```

```
# [1] "abc"
```

```r
grep("^[[:alnum:]]+$", x, value=TRUE)             # (d)
```

```
# [1] "123" "abc"
```

Ex 107: Lookahead/Lookbehind

Problem

- **(a)** Extract numbers preceded by `\$`: `c("\$100","200","\$300")`
- **(b)** Find files NOT ending in `.txt`: `c("a.txt","b.png","c.txt","d.jpg")`
- **(c)** Explain why `perl=TRUE` is needed

Sol 107

```r
x <- c("$100","200","$300")
regmatches(x, gregexpr("(?<=\\$)\\d+", x, perl=TRUE)) # (a)
```

```
# [[1]]
# [1] "100"
#
# [[2]]
# character(0)
#
# [[3]]
# [1] "300"
```

```r
y <- c("a.txt","b.png","c.txt","d.jpg")
grep("\\.(?!txt$)", y, value=TRUE, perl=TRUE)      # (b)
```

```
# [1] "b.png" "d.jpg"
```

```r
# (c) lookahead/lookbehind are Perl-compatible regex
# extensions not available in base (POSIX) regex
```

Ex 108: `regexec` — Structured Extraction

Problem

`dates <- c("2024-01-15 14:30", "2023-12-31 09:00")`.

- **(a)** Write pattern to capture year, month, day, hour, minute separately
- **(b)** Extract with `regexec` + `regmatches`
- **(c)** Build a data frame from the results

Sol 108

```r
dates <- c("2024-01-15 14:30", "2023-12-31 09:00")
pat <- "(\\d{4})-(\\d{2})-(\\d{2}) (\\d{2}):(\\d{2})"
m <- regexec(pat, dates)
parts <- regmatches(dates, m)                          # (a)(b)
do.call(rbind, lapply(parts, \(p)
  setNames(p[-1], c("Y","M","D","h","m"))))            # (c)
```

```
#      Y      M    D    h    m
# [1,] "2024" "01" "15" "14" "30"
# [2,] "2023" "12" "31" "09" "00"
```

Ex 109: camelCase snake_case

Problem

- **(a)** Write `camel\_to\_snake(s)` converting `"camelCase"` → `"camel\_case"`
- **(b)** Write `snake\_to\_camel(s)` converting `"snake\_case"` → `"snakeCase"`
- **(c)** Test both on `"myHTTPRequest"` / `"my\_http\_request"`

Sol 109

```r
camel_to_snake <- function(s) tolower(gsub("([A-Z])","_\\1",s))
camel_to_snake("camelCase")                          # (a)
```

```
# [1] "camel_case"
```

```r
camel_to_snake("myHTTPRequest")
```

```
# [1] "my_h_t_t_p_request"
```

```r
snake_to_camel <- function(s) {
  parts <- strsplit(s, "_")[[1]]
  paste0(parts[1], paste0(toupper(substring(parts[-1],1,1)),
         substring(parts[-1],2), collapse=""))
}
snake_to_camel("my_http_request")                    # (b)(c)
```

```
# [1] "myHttpRequest"
```

Ex 110: Clean Column Names

Problem

`cols <- c("First Name", "Last.Name", "Age (years)", "Income$")`.

- **(a)** Replace all non-alphanumeric chars with _
- **(b)** Remove leading/trailing _
- **(c)** Collapse consecutive _ to single _
- **(d)** Convert to lowercase

Sol 110

```r
cols <- c("First Name","Last.Name","Age (years)","Income$")
x <- gsub("[^a-zA-Z0-9]", "_", cols)              # (a)
x <- gsub("^_+|_+$", "", x)                       # (b)
x <- gsub("_+", "_", x)                           # (c)
tolower(x)                                         # (d)
```

```
# [1] "first_name" "last_name"  "age_years"  "income"
```

Date/Time, I/O, Error Handling, Advanced (Ex 111–120)

Ex 111: Date Arithmetic

Problem

- **(a)** Create today's date with `Sys.Date()`. Add 30 days
- **(b)** Days between `2024-01-01` and `2024-12-31`
- **(c)** Generate monthly sequence Jan–Jun 2024
- **(d)** Extract weekday name and month name from a date

Sol 111

```r
today <- Sys.Date(); today + 30                    # (a)
```

```
# [1] "2026-04-18"
```

```r
as.Date("2024-12-31") - as.Date("2024-01-01")     # (b) 365
```

```
# Time difference of 365 days
```

```r
seq(as.Date("2024-01-01"), as.Date("2024-06-01"),
    by="month")                                    # (c)
```

```
# [1] "2024-01-01" "2024-02-01" "2024-03-01" "2024-04-01" "2024-05-01"
# [6] "2024-06-01"
```

```r
d <- as.Date("2024-07-15")
weekdays(d); months(d); format(d, "%A %B %d, %Y") # (d)
```

```
# [1] " "

# [1] " "

# [1] "    15, 2024"
```

Ex 112: `POSIXct` and Timezones

Problem

- **(a)** Create `as.POSIXct("2024-01-15 14:30:00", tz="Asia/Taipei")`
- **(b)** Extract hour, minute via `POSIXlt`
- **(c)** Difference between two timestamps in hours
- **(d)** Format with `strftime`

Sol 112

```r
t1 <- as.POSIXct("2024-01-15 14:30:00", tz="Asia/Taipei") # (a)
lt <- as.POSIXlt(t1); lt$hour; lt$min             # (b)
```

```
# [1] 14

# [1] 30
```

```r
t2 <- as.POSIXct("2024-01-15 18:45:00", tz="Asia/Taipei")
difftime(t2, t1, units="hours")                    # (c)
```

```
# Time difference of 4.25 hours
```

```r
strftime(t1, "%Y/%m/%d %H:%M %Z")                 # (d)
```

```
# [1] "2024/01/15 14:30 CST"
```

Ex 113: Binary I/O

Problem

- **(a)** `charToRaw("Hello")` — what type?
- **(b)** Convert back with `rawToChar`
- **(c)** `as.integer(charToRaw("ABC"))` — what are the byte values?
- **(d)** Write raw bytes to a temp file with `writeBin` and read back

Sol 113

```r
r <- charToRaw("Hello"); r; typeof(r)              # (a) raw
```

```
# [1] 48 65 6c 6c 6f
```

```
# [1] "raw"
```

```r
rawToChar(r)                                        # (b) "Hello"
```

```
# [1] "Hello"
```

```r
as.integer(charToRaw("ABC"))                        # (c) 65 66 67
```

```
# [1] 65 66 67
```

```r
tmp <- tempfile()                                   # (d)
writeBin(charToRaw("test123"), tmp)
rawToChar(readBin(tmp, "raw", 20))
```

```
# [1] "test123"
```

Ex 114: `readline` and `interactive`

Problem

- **(a)** Write a function that uses `readline` to ask for user's name (use `eval=FALSE`)
- **(b)** Use `interactive()` to branch: interactive $\rightarrow$ readline, batch $\rightarrow$ default
- **(c)** Why does `readline` always return character?

Sol 114

```r
ask_name <- function() {
  if (interactive()) {
    name <- readline("Your name: ")
  } else {
    name <- "default_user"
  }
  cat("Hello,", name, "\n")
}
# (c) readline reads raw text from console; R has no way
# to know the intended type, so it always returns character
```

Ex 115: `system2`

Problem

- **(a)** Run `system2("echo", args=c("hello","world"))`
- **(b)** Capture output with `stdout=TRUE`
- **(c)** Run in background with `wait=FALSE`
- **(d)** Cross-platform check with `.Platform\$OS.type`

Sol 115

```r
system2("echo", args=c("hello","world"))            # (a)
out <- system2("date", stdout=TRUE); out            # (b)
system2("sleep", args="1", wait=FALSE)              # (c)
if (.Platform$OS.type == "unix") {                  # (d)
  system2("uname", args="-a", stdout=TRUE)
}
```

Ex 116: S3 Classes

Problem

- **(a)** Create S3 class `Circle`: `list(radius=r)` with `class="Circle"`
- **(b)** Write `print.Circle` and `area.Circle`
- **(c)** Create instance; dispatch `print` and `area`

Sol 116

```r
Circle <- function(r) structure(list(radius=r), class="Circle")
print.Circle <- function(x, ...)
  cat(sprintf("Circle(r=%.2f)\n", x$radius))
area <- function(x, ...) UseMethod("area")
area.Circle <- function(x, ...) pi * x$radius^2
c1 <- Circle(5); c1                                  # (a)(b)(c)
```

```
# Circle(r=5.00)
```

```r
area(c1)
```

```
# [1] 78.53982
```

Ex 117: Formula and `lm`

Problem

- **(a)** `f <- y \textasciitilde\{\ x}`; show `class`, `typeof`, `all.vars`
- **(b)** Fit `lm(dist \textasciitilde\{\ speed, data=cars)}`. Extract coefficients
- **(c)** Plot data with regression line

Sol 117

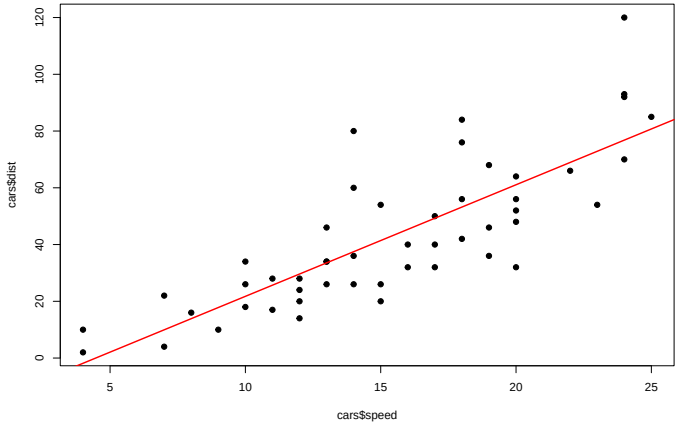
```r
f <- y ~ x; class(f); typeof(f); all.vars(f)      # (a)
```

```
# [1] "formula"

# [1] "language"

# [1] "y" "x"
```

```r
model <- lm(dist ~ speed, data=cars)
coef(model)                                       # (b)
```

```
# (Intercept)       speed
#  -17.579095    3.932409
```

```r
plot(cars$speed, cars$dist, pch=19)               # (c)
abline(model, col="red", lwd=2)
```



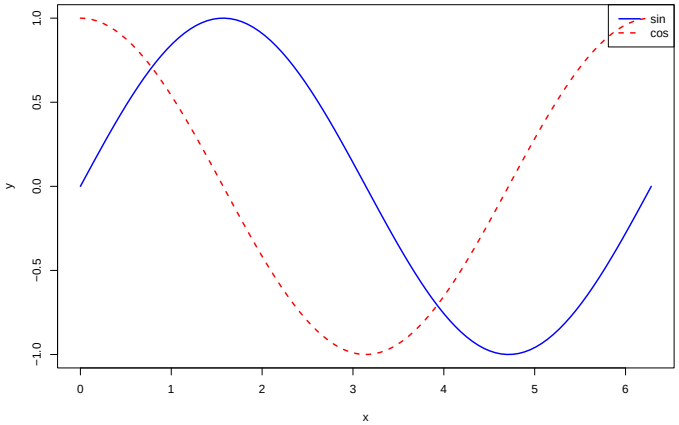
Ex 118: Plotting

Problem

- **(a)** Plot `sin(x)` and `cos(x)` for `x` from 0 to 2π with legend
- **(b)** `par(mfrow=c(1,2))`: histogram and boxplot side by side
- **(c)** Add annotation with `text()` and horizontal line with `abline(h=...)`

Sol 118

```
x <- seq(0, 2*pi, length=100)                    # (a)
plot(x, sin(x), type="l", col="blue", lwd=2, ylab="y")
lines(x, cos(x), col="red", lwd=2, lty=2)
legend("topright", c("sin","cos"), col=c("blue","red"),
       lty=1:2, lwd=2)
```



Ex 119: Random Number Generation

Problem

- **(a)** `set.seed(42)` then generate 5 uniform, 5 normal, 10 Bernoulli
- **(b)** Simulate 1000 dice rolls and tabulate
- **(c)** Estimate π via Monte Carlo: sample `(x,y)` uniform in $[0,1]^2$, count proportion inside unit circle quadrant

Sol 119

```r
set.seed(42)
runif(5); rnorm(5); rbinom(10, 1, 0.5)            # (a)
```

```
# [1] 0.9148060 0.9370754 0.2861395 0.8304476 0.6417455

# [1]  0.04788474 -1.10459944  0.53902380  0.58020632 -0.65750284

#  [1] 1 1 0 0 1 1 0 1 1 0
```

```r
table(sample(1:6, 1000, replace=TRUE))             # (b)
```

```
#
#   1   2   3   4   5   6
# 169 191 163 157 153 167
```

```r
n <- 1e5; x <- runif(n); y <- runif(n)            # (c)
4 * mean(x^2 + y^2 <= 1)  #
```

```
# [1] 3.14176
```

Ex 120: Options and Benchmarking

Problem

- **(a)** Save current `digits` option, set to 15, print `pi`, restore
- **(b)** Time `crossprod(x)` for `x <- matrix(rnorm(1e6),1000)` with `system.time`
- **(c)** Use `proc.time` for manual timing of `sum(1:1e7)`

Sol 120

```r
old <- options(digits=15); pi                          # (a)
```

```
# [1] 3.14159265358979
```

```r
options(old)
system.time({                                          # (b)
  x <- matrix(rnorm(1e6), 1000, 1000)
  y <- crossprod(x)
})
```

```
#    user  system elapsed
#   0.090   0.075   0.038
```

```r
pt <- proc.time(); s <- sum(1:1e7); proc.time()-pt # (c)
```

```
#    user  system elapsed
#   0.000   0.014   0.001
```